

MÉMOIRE DE PROJET DE FIN D'ETUDES

Présenté pour obtenir le
DIPLÔME NATIONAL D'INGÉNIEUR
Spécialité : GENIE ÉLECTRIQUE

Par
Raslen ZIADI
Né(e) le : 16/11/2002 à Nabul

*Conception et développement d'un framework d'automatisation pour
la validation du boot sécurisé.*

Soutenu le 18/07/2026, devant le jury d'examen :

Mr.Ramzi HADDAJI	Président
Mr.Anis SAKLY	Membre
Mr.Maher KOURAICHI	Encadrant académique
Mr.Abdelhak DAOUDI	Encadrant professionnel

Dédicaces

Je tiens tout d'abord à exprimer ma profonde gratitude à mes chers parents pour leur soutien et leurs encouragements constants tout au long de mon parcours académique.

*J'adresse également cette reconnaissance à ma chère mère, **Sihem Batnini**, pour son amour inconditionnel, sa tendresse, ses sacrifices silencieux et son soutien constant. Ma reconnaissance envers toi est infinie.*

À mon père, pour son soutien, ses encouragements et sa présence précieuse.

À mes amis les plus chers, pour les moments de complicité, les rires partagés et leur présence tout au long de mon parcours.

À toutes les personnes qui m'ont aidé, soutenu ou inspiré, de près ou de loin.

Que ce modeste travail soit à la hauteur de votre confiance.

Raslen Ziadi

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude envers Madame **Mouna BEN RHOUMA** pour m'avoir accueilli au sein de son équipe.

Mes sincères remerciements s'adressent également à Monsieur **Abdelhak DAOUDI**, mon superviseur chez STMicroelectronics, pour sa disponibilité constante et ses conseils précieux et éclairés tout au long de ce stage.

Je souhaite aussi témoigner ma reconnaissance à l'ensemble des membres de l'équipe « **Test et Validation** » pour leur accueil chaleureux et leur aide précieuse. Un remerciement particulier à Monsieur **Maher KOURAICHI**, mon enseignant à l'École Nationale d'Ingénieurs de Monastir, pour son soutien indéfectible.

Enfin, j'exprime ma gratitude aux membres du jury pour l'honneur qu'ils me font en évaluant mon travail, ainsi qu'aux enseignants de l'École Nationale d'Ingénieurs de Monastir pour leur engagement à assurer une formation d'excellence au sein de cette institution.

Table des matières

Dédicaces	i
Remerciements	ii
Table des matières	iii
Table des figures	vii
Liste des tableaux	ix
Liste des acronymes et abréviations	x
Introduction générale	xii
1 Contexte général du projet	1
1.1 Introduction	1
1.2 Présentation de l'entreprise	1
1.2.1 Bref historique	2
1.2.2 Produits et solutions de STMicroelectronics	2
1.2.3 STMicroelectronics Tunisie	3
1.3 Contexte du projet	4
1.3.1 Présentation générale du projet	4
1.3.2 Problématique	4
1.3.3 Objectifs du projet	5
1.4 Méthodologie de travail	5
1.4.1 Pourquoi une démarche itérative ?	5
1.4.2 Limites d'une approche en cascade	6
1.4.3 Apport des méthodes agiles	6
1.4.3.1 Scrum	6
1.4.3.2 Kanban	7
1.4.3.3 Comparaison entre Scrum et Kanban	8
1.4.4 Méthodologie adoptée	8
1.5 Conclusion	8

2	Étude théorique	10
2.1	Introduction	10
2.2	Fondamentaux des tests logiciels	10
2.2.1	Classification des tests	11
2.2.1.1	Approches fondamentales des tests logiciels	11
2.2.1.2	Hierarchie des niveaux de test	12
2.2.2	Automatisation des tests	14
2.2.2.1	Définition	14
2.2.2.2	Tests manuels	14
2.2.2.3	Tests automatisés	14
2.2.2.4	Avantages	15
2.3	Microcontrôleurs STM32 et sécurité embarquée	16
2.3.1	Présentation des microcontrôleurs STM32	16
2.3.1.1	Architecture ARM Cortex	17
2.3.1.2	Interfaces multiples	17
2.3.1.3	Mémoire embarquée des STM32	20
2.3.2	Mécanismes de sécurité des STM32	21
2.3.2.1	Secure Boot	21
2.3.2.2	TrustZone	22
2.3.2.3	Product State	23
2.3.2.4	Hide Protection (HDP)	24
2.3.3	Scénarios de boot sécurisé STM32	24
2.3.3.1	ST IROT	25
2.3.3.2	OEM IROT	25
2.3.3.3	ST IROT / OEMuROT	25
2.4	Environnement matériel et logiciel	25
2.4.1	Environnement matériel	25
2.4.1.1	Cartes STM32	25
2.4.1.2	ST-LINK	26
2.4.1.3	UART	28
2.4.1.4	Bootloader	28
2.4.2	Environnement logiciel	29
2.4.2.1	STM32CubeMX	29
2.4.2.2	STM32CubeIDE	29
2.4.2.3	STM32CubeProgrammer	30
2.4.2.4	Trusted Package Creator (TPC)	32
2.4.2.5	Robot Framework	32
2.4.2.6	Python : Langage de programmation et environnement de développement intégré (IDE) PyCharm	33

2.4.2.7	Git : système de gestion de versions	34
2.4.2.8	Tkinter	35
2.4.2.9	SikuliX	35
2.5	Intelligence artificielle appliquée à l'automatisation des tests	35
2.5.1	Intelligence artificielle et modèles LLM	35
2.5.1.1	Concepts généraux	35
2.5.1.2	Fonctionnement des LLM	35
2.5.2	Diagnostic assisté par IA	36
2.5.2.1	Analyse des logs	36
2.5.2.2	Classification des erreurs	36
2.5.2.3	Génération de recommandations	36
2.5.3	Apport de l'IA dans le projet	36
2.5.3.1	Analyse automatique des incidents	36
2.5.3.2	Aide au débogage	36
2.5.3.3	Génération de rapports	37
2.6	Conclusion	37
3	Réalisation et évaluation de la solution	38
3.1	Introduction	38
3.2	Configuration matérielle	38
3.2.1	Cartes cibles utilisées	39
3.2.2	Carte supervisor	39
3.2.3	Interfaces de connexion et câblage	40
3.3	Processus des tests automatiques et architecture globale	41
3.3.1	Cadre de la conception des tests	41
3.3.1.1	Architecture d'exécution et déroulement des scénarios automatisés	42
3.4	Mise en œuvre de la solution	45
3.4.1	Lancement et pilotage des scénarios	45
3.4.1.1	Paramétrage des scénarios et des cartes	46
3.4.2	Exécution des scénarios automatisés	46
3.4.2.1	Préparation du projet et compilation	47
3.4.2.2	Provisioning et programmation	47
3.4.2.3	Reset matériel via la carte supervisor	48
3.4.2.4	Capture UART et validation du boot	49
3.4.2.5	Exécution de la full regression	49
3.4.3	Gestion des artefacts et post-traitement	49
3.4.3.1	Collecte des logs et preuves de validation	51
3.4.3.2	Construction de l'incident JSON	52

3.4.3.3	Diagnostic local et enrichissement par IA	53
3.4.3.4	Génération du rapport Excel	55
3.5	Résultats obtenus et discussion	57
3.6	Considérations finales et limitations	58
3.7	Conclusion	59
Conclusion générale		60
Bibliographie		62

Table des figures

1.1	Présence mondiale de STMicroelectronics	2
1.2	Gamme de produits de STMicroelectronics	3
1.3	Modèle en cascade (Waterfall)	6
1.4	Méthode Scrum	7
1.5	Méthode Kanban	7
2.1	Tests boîte noire	11
2.2	Tests boîte blanche	12
2.3	Hiérarchie des niveaux de test	13
2.4	Processus d'un test manuel	14
2.5	Processus d'un test automatisé	15
2.6	Composants internes d'un microcontrôleur typique	16
2.7	Exemple de câblage d'une liaison I2C	18
2.8	Exemple de câblage d'une liaison UART	18
2.9	Exemple de câblage d'une liaison SPI	19
2.10	Exemple de câblage d'une liaison USB	19
2.11	Organisation mémoire typique d'un microcontrôleur	20
2.12	Organisation simplifiée d'une zone Flash	21
2.13	Chaîne Secure Boot : racine de confiance, vérification du firmware et lan- cement de l'application	22
2.14	Séparation fonctionnelle entre les domaines Secure et Non-Secure avec TrustZone	23
2.15	Cycle de vie du Product State	24
2.16	Différentes familles de cartes STM32	26
2.17	ST-LINK	27
2.18	Connexion ST-LINK	27
2.19	Interface de STM32CubeMX	29
2.20	Interface de STM32CubeIDE	30
2.21	Interface graphique de STM32CubeProgrammer	31
2.22	Interface en ligne de commande de STM32CubeProgrammer	31
2.23	Interface de STM32TrustedPackageCreator	32
2.24	Robot Framework	33

2.25	Architecture d'un système de gestion de versions distribué Git	34
3.1	Cartes cibles STM32 utilisées dans le cadre du projet	39
3.2	Carte supervisor STM32U599J-DK	40
3.3	Architecture des connexions matérielles du banc de validation	41
3.4	Logique de conception du framework d'automatisation	42
3.5	Déroulement global d'un scénario de validation automatisé	44
3.6	Interface graphique de pilotage du framework	45
3.7	Exemple de fichier de configuration dans le framework	46
3.8	Préparation du projet et phase de compilation	47
3.9	Phase de provisioning et de programmation	48
3.10	Principe du reset matériel de la carte cible via la carte supervisor	48
3.11	Capture UART	49
3.12	Chaîne de post-traitement après l'exécution d'un scénario	51
3.13	Organisation des artefacts et des logs générés	52
3.14	Construction de l'incident JSON à partir des artefacts du run	53
3.15	Rapport IA généré	54
3.16	Exemple de diagnostic IA pour un run échoué (scénario ST_IROT)	55
3.17	Rapport Excel généré par le framework après post-traitement	56
3.18	Extrait du rapport Excel pour un run échoué	57
3.19	Comparaison du temps de validation d'un scénario ST_IROT : validation manuelle vs validation automatique	58

Liste des tableaux

1.1	Comparaison entre Scrum et Kanban	8
2.1	Modes de démarrage selon l'état des broches de boot	28

Liste des acronymes et abréviations

API	Application Programming Interface
CLI	Command Line Interface
CPU	Central Processing Unit
DK	Discovery Kit
GUI	Graphical User Interface
GPIO	General Purpose Input Output
HDP	Hide Protection
IA	Intelligence Artificielle
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LLM	Large Language Model
MCU	Microcontroller Unit
NRST	Reset Pin
OEM	Original Equipment Manufacturer
OEMuROT	OEM Updatable Root Of Trust
RDP	Readout Protection
RoT	Root of Trust
ST-LINK	STMicroelectronics Debug and Programming Interface
STiROT	ST Immutable Root Of Trust
SWD	Serial Wire Debug
TFM	Trusted Firmware-M

TPC	Trusted Package Creator
TZEN	TrustZone Enable
UART	Universal Asynchronous Receiver Transmitter
USB	Universal Serial Bus
USART	Universal Synchronous Asynchronous Receiver Transmitter
VCP	Virtual COM Port
XML	Extensible Markup Language

Introduction générale

Avec l'évolution continue des systèmes embarqués, les exigences en matière de sécurité sont devenues de plus en plus importantes. Les microcontrôleurs modernes ne se limitent plus à exécuter des fonctions de contrôle ou de traitement classiques ; ils doivent également garantir l'intégrité du code exécuté, protéger les ressources sensibles et assurer un démarrage fiable dans des environnements parfois critiques. Dans ce contexte, les architectures STM32 intègrent aujourd'hui plusieurs mécanismes avancés, tels que le *Secure Boot*, le *TrustZone* ou encore les opérations de *provisioning*, afin de renforcer la sécurité dès les premières étapes de l'exécution.

La validation de ces mécanismes représente toutefois un défi important. Elle mobilise plusieurs outils logiciels, différents environnements de développement, ainsi que des cartes matérielles sur lesquelles les scénarios doivent être exécutés et vérifiés. En pratique, la succession des opérations à réaliser — génération ou ouverture de projet, compilation, provisioning, programmation, reset matériel, capture UART et analyse des résultats — rend les campagnes de test longues, sensibles aux erreurs de manipulation et difficiles à reproduire de manière homogène lorsqu'elles sont effectuées manuellement.

Dans un tel contexte, l'automatisation apparaît comme une nécessité plutôt qu'un simple confort. Elle permet non seulement d'accélérer l'exécution des scénarios de validation, mais aussi d'en renforcer la répétabilité, de centraliser les preuves de test et d'améliorer la qualité de l'analyse en cas d'échec. À cela s'ajoute aujourd'hui l'apport de l'intelligence artificielle, qui ouvre de nouvelles possibilités pour interpréter les logs, rapprocher les incidents similaires et assister le diagnostic de manière plus structurée.

C'est dans cette perspective que s'inscrit ce projet de fin d'études, réalisé au sein de STMicroelectronics. Le travail présenté dans ce rapport porte sur la conception et la mise en place d'un framework d'automatisation destiné à la validation de scénarios de boot sécurisé sur cartes STM32. L'objectif n'est pas uniquement d'automatiser une suite d'actions, mais de proposer une véritable chaîne de validation, capable d'orchestrer les scénarios, de collecter les artefacts de preuve, de consolider les incidents et de produire un diagnostic exploitable.

Le présent rapport est structuré en trois chapitres. Le premier chapitre présente le cadre général du projet, l'entreprise d'accueil ainsi que la problématique ayant motivé ce travail. Le deuxième chapitre expose les fondements théoriques, les mécanismes de sécurité étudiés et l'environnement technologique mobilisé dans le cadre du projet. Enfin, le

troisième chapitre détaille la réalisation du framework, les principaux choix d'implémentation, ainsi que les résultats obtenus et les limites identifiées.

Contexte général du projet

1.1 Introduction

Ce premier chapitre a pour objectif de situer le projet dans son environnement général, à la fois industriel et technique. Il présente d’abord l’entreprise STMicroelectronics, son positionnement à l’échelle internationale ainsi que l’importance de son implantation en Tunisie. Il introduit ensuite le contexte du stage, la problématique à laquelle nous avons été confrontés et les objectifs fixés au début du projet. Enfin, il décrit la démarche méthodologique adoptée pour conduire le travail dans un cadre structuré tout en restant adapté à un environnement technique évolutif.

1.2 Présentation de l’entreprise

STMicroelectronics est un acteur mondial majeur dans le domaine des semi-conducteurs. Issue de la fusion entre des entités françaises et italiennes, l’entreprise conçoit et fabrique des composants électroniques destinés à des secteurs variés, tels que l’automobile, l’industrie, les télécommunications, les objets connectés ou encore l’électronique grand public. Grâce à la diversité de son offre et à sa forte capacité d’innovation, STMicroelectronics occupe aujourd’hui une place importante dans l’écosystème mondial des technologies embarquées.

Cotée en bourse en Europe et aux États-Unis, l’entreprise a son siège social à Plan-les-Ouates, près de Genève, en Suisse [1]. Elle emploie plusieurs dizaines de milliers de collaborateurs à travers le monde, dont une part significative est dédiée aux activités de recherche et développement. Cette présence internationale lui permet d’assurer à la fois une proximité avec ses clients et une forte réactivité face aux évolutions du marché [1].

La figure 1.1 illustre la présence mondiale de STMicroelectronics à travers ses principaux sites et son rayonnement international.

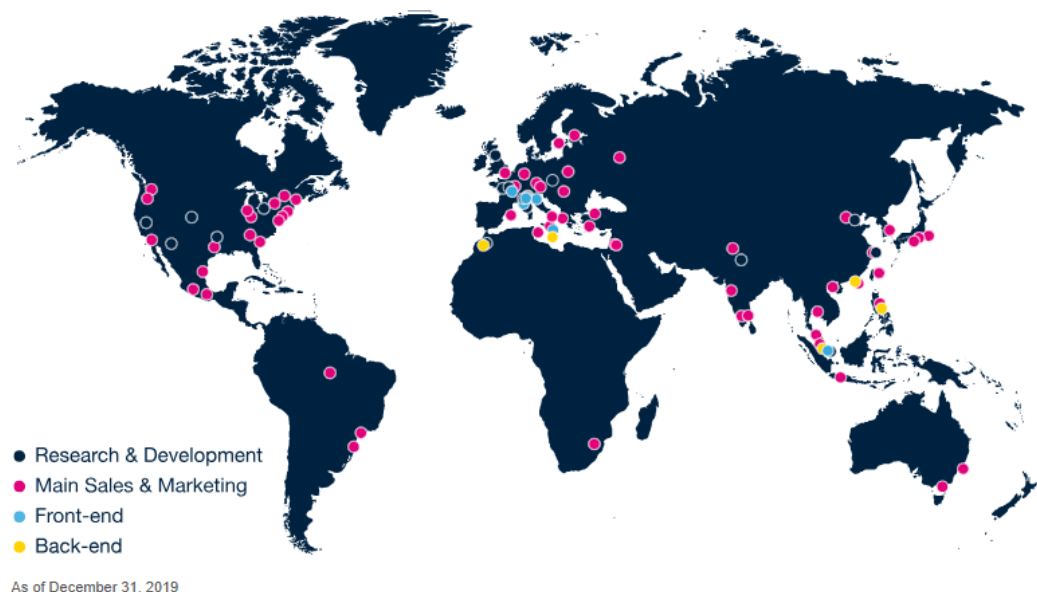


Fig. 1.1 : Présence mondiale de STMicroelectronics

1.2.1 Bref historique

STMicroelectronics a vu le jour en 1987 à la suite de la fusion entre l'entreprise italienne SGS (*Società Generale Semiconduttori*) et la société française Thomson Semiconducteurs, filiale du groupe Thomson. Dans un premier temps, l'entreprise a porté le nom de SGS-Thomson avant d'adopter, en 1998, son nom actuel après le retrait de Thomson de son capital.

Depuis sa création, la société a connu une croissance continue et a su s'imposer comme un acteur de référence dans l'univers des semi-conducteurs, notamment grâce à la diversité de ses produits, à sa stratégie d'innovation et à sa présence dans de nombreux domaines applicatifs.

1.2.2 Produits et solutions de STMicroelectronics

Le portefeuille de STMicroelectronics couvre un large éventail de produits électroniques répondant aux besoins de nombreux secteurs industriels. L'entreprise conçoit notamment des microcontrôleurs, parmi lesquels la famille STM32 occupe une place centrale, ainsi que des circuits intégrés de puissance, des capteurs intelligents et différents composants discrets tels que les diodes, transistors et régulateurs.

La famille STM32, introduite en 2007, s'est imposée comme l'une des références majeures du marché des microcontrôleurs 32 bits. Elle est aujourd'hui largement utilisée dans des applications embarquées allant des systèmes industriels aux objets connectés, en passant par l'automobile et les systèmes critiques.

Au-delà de l'aspect matériel, STMicroelectronics propose également un écosystème

complet d'outils de développement. Celui-ci comprend des environnements comme STM32CubeIDE, des outils de configuration tels que STM32CubeMX, des bibliothèques logicielles, des cartes d'évaluation et divers utilitaires destinés à faciliter le développement, la validation et l'intégration des solutions embarquées.

La figure 1.2 présente un aperçu de la diversité des produits développés par STMicroelectronics.

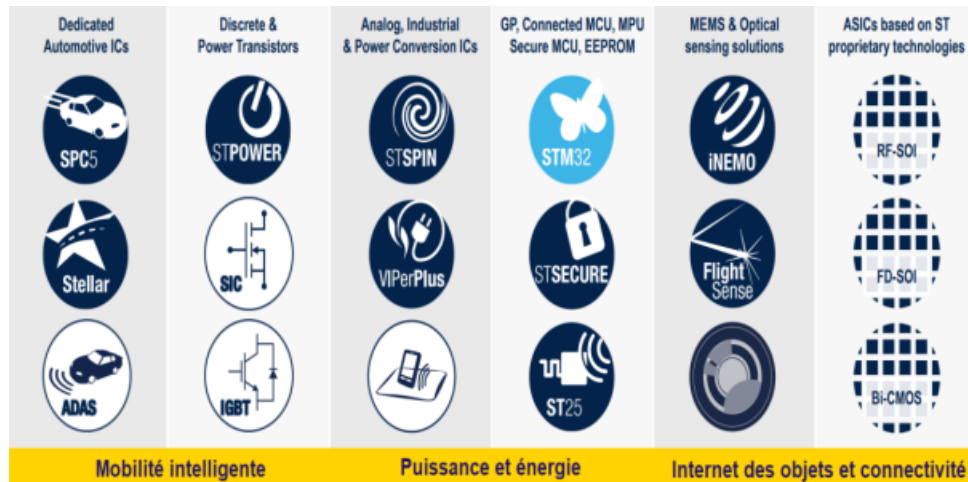


Fig. 1.2 : Gamme de produits de STMicroelectronics

1.2.3 STMicroelectronics Tunisie

Parmi les différents sites de STMicroelectronics dans le monde, le centre implanté en Tunisie occupe une place importante dans les activités de recherche et développement de l'entreprise. Situé à la Cité des Technologies des Communications d'El Ghazala, ce site a été inauguré en décembre 2001 avec une équipe initiale restreinte. Au fil des années, il s'est progressivement développé pour devenir un centre reconnu dans la conception et le développement de solutions embarquées, en particulier dans le domaine des microcontrôleurs [2].

Le site de Tunis intervient notamment dans les activités de la division *General-Purpose Microcontroller* (GPM), rattachée au groupe *Microcontrollers and Digital ICs Group* (MDG). Cette division est impliquée dans la conception de microcontrôleurs ainsi que dans le développement de solutions logicielles et d'outils associés.

Le présent projet a été réalisé au sein de l'équipe *Software Development Tools*, plus précisément dans une équipe orientée test et validation. Ce cadre a permis de travailler sur des problématiques concrètes liées à l'automatisation de la validation, dans un environnement industriel exigeant et en lien direct avec les outils de l'écosystème STM32.

1.3 Contexte du projet

1.3.1 Présentation générale du projet

Ce projet de fin d'études s'inscrit dans un besoin d'automatisation des campagnes de validation associées aux scénarios de boot sécurisé sur cartes STM32. Dans le cadre de ce travail, l'objectif principal était de concevoir et de mettre en œuvre une solution capable d'exécuter automatiquement des scénarios de validation qui, jusque-là, restaient en grande partie manuels et donc sensibles aux erreurs de manipulation.

Le projet porte sur plusieurs flows de sécurisation du démarrage, notamment les scénarios `ST_IROT`, `OEM_IROT`, `STIROT_OEMUROT`, ainsi que certaines variantes basées sur des projets template. L'automatisation développée devait prendre en charge différentes étapes du processus, telles que l'ouverture ou la génération du projet, la compilation, le provisioning, la programmation des cartes, le déclenchement du reset matériel, la capture UART et la validation des résultats.

Ce travail ne se limite pas à l'exécution automatique d'un enchaînement de scripts. Il vise plus largement à structurer toute une chaîne de validation, depuis le lancement du scénario par l'utilisateur jusqu'à la génération d'un rapport final contenant les preuves de test et une aide au diagnostic.

1.3.2 Problématique

Dans leur forme initiale, les scénarios de validation du boot sécurisé reposaient sur une suite d'opérations manuelles mobilisant plusieurs outils STM32, différents paramètres matériels et plusieurs phases de vérification. En pratique, cette manière de travailler présentait plusieurs limites.

D'abord, le temps d'exécution global était relativement élevé, car chaque étape devait être lancée, supervisée puis vérifiée manuellement. Ensuite, la répétabilité des tests n'était pas garantie de manière homogène, notamment en raison des différences possibles entre les postes de travail, des temps d'attente variables, ou encore des erreurs de manipulation. Enfin, lorsqu'un incident survenait, l'analyse était souvent plus difficile qu'attendu, car les logs n'étaient pas toujours centralisés et les preuves de validation pouvaient être dispersées.

À mesure que les scénarios de sécurité devenaient plus riches et plus nombreux, cette approche manuelle montrait donc ses limites. Il devenait nécessaire de mettre en place une solution d'automatisation capable d'apporter à la fois plus de fiabilité, plus de traçabilité et une meilleure capacité de diagnostic.

1.3.3 Objectifs du projet

L’objectif général du projet est de développer un framework d’automatisation dédié à la validation de scénarios de boot sécurisé sur microcontrôleurs STM32. À travers ce travail, plusieurs objectifs spécifiques ont été définis :

- automatiser l’exécution des principaux scénarios de validation liés au secure boot ;
- réduire les manipulations manuelles sur les outils STM32CubeMX, STM32CubeIDE et STM32CubeProgrammer ;
- rendre les exécutions de tests plus fiables et plus reproductibles ;
- centraliser les logs, les preuves de validation et les artefacts générés ;
- utiliser la sortie UART comme preuve principale du bon déroulement du boot ;
- enrichir l’analyse grâce à la collecte d’informations de sécurité complémentaires ;
- faciliter l’identification des incidents à travers un diagnostic structuré ;
- générer un rapport final exploitable pour le suivi et l’analyse des campagnes de test.

Au-delà de l’automatisation pure, le projet vise également à améliorer la qualité globale du processus de validation en proposant un cadre plus structuré et mieux adapté à un contexte industriel.

1.4 Méthodologie de travail

1.4.1 Pourquoi une démarche itérative ?

Le développement d’un framework de validation dans un contexte embarqué ne suit pas toujours une progression strictement linéaire. En effet, les besoins évoluent au fil des essais, certaines contraintes techniques n’apparaissent qu’au moment de l’intégration sur matériel réel, et plusieurs ajustements doivent souvent être réalisés après l’observation des premiers résultats.

Dans ce type de projet, une méthodologie entièrement séquentielle devient rapidement difficile à appliquer. Le travail demande au contraire une capacité d’adaptation, des retours fréquents sur les choix réalisés, ainsi qu’une progression par améliorations successives.

1.4.2 Limites d’une approche en cascade

Le modèle en cascade (*Waterfall*) repose sur une succession de phases bien distinctes : analyse, conception, développement, test puis validation. Cette approche est pertinente lorsque les besoins sont stables dès le départ et que les évolutions sont limitées.

Cependant, dans le cadre de ce projet, plusieurs éléments rendaient ce modèle peu adapté : l’évolution progressive des scénarios à automatiser, l’interaction constante avec des outils externes, la dépendance au matériel réel et la nécessité d’ajuster la solution au fur et à mesure des tests. Une organisation trop rigide aurait donc limité la réactivité et ralenti l’avancement du projet.

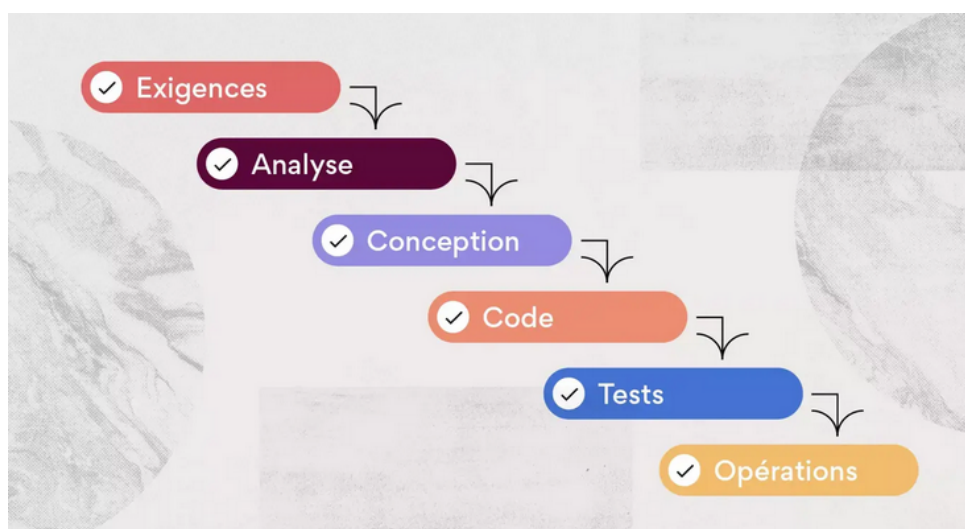


Fig. 1.3 : Modèle en cascade (Waterfall)

1.4.3 Apport des méthodes agiles

Les approches agiles offrent un cadre plus flexible, particulièrement adapté aux projets où l’expérimentation, les retours fréquents et l’évolution progressive des besoins occupent une place importante. Elles permettent d’avancer par cycles courts, d’évaluer régulièrement les résultats obtenus et d’ajuster les priorités lorsque cela devient nécessaire.

Dans le cadre de ce projet, cette logique s’est révélée pertinente, car le framework a été enrichi progressivement : ajout de nouveaux scénarios, amélioration des bibliothèques métier, stabilisation des mécanismes de reset, consolidation du diagnostic et correction de problèmes liés à l’environnement matériel.

1.4.3.1 Scrum

Scrum est un cadre agile structuré autour d’itérations courtes appelées *sprints*. Il définit un ensemble de rôles, de réunions et de livrables permettant d’organiser le travail

de manière formelle. Cette approche convient bien lorsque les objectifs sont clairement priorités et qu'une équipe dédiée peut fonctionner selon ce rythme.

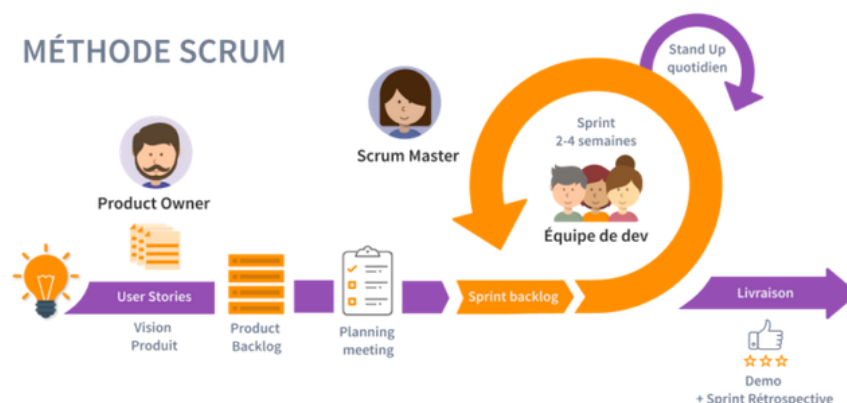


Fig. 1.4 : Méthode Scrum

1.4.3.2 Kanban

Kanban est une méthode plus souple, fondée sur la visualisation du flux de travail et la gestion progressive des tâches. Elle permet de suivre l'état d'avancement des activités en continu, sans imposer de cycles fixes. Cette approche est particulièrement intéressante lorsque les priorités évoluent régulièrement, ou lorsque certaines tâches dépendent fortement de contraintes techniques externes.

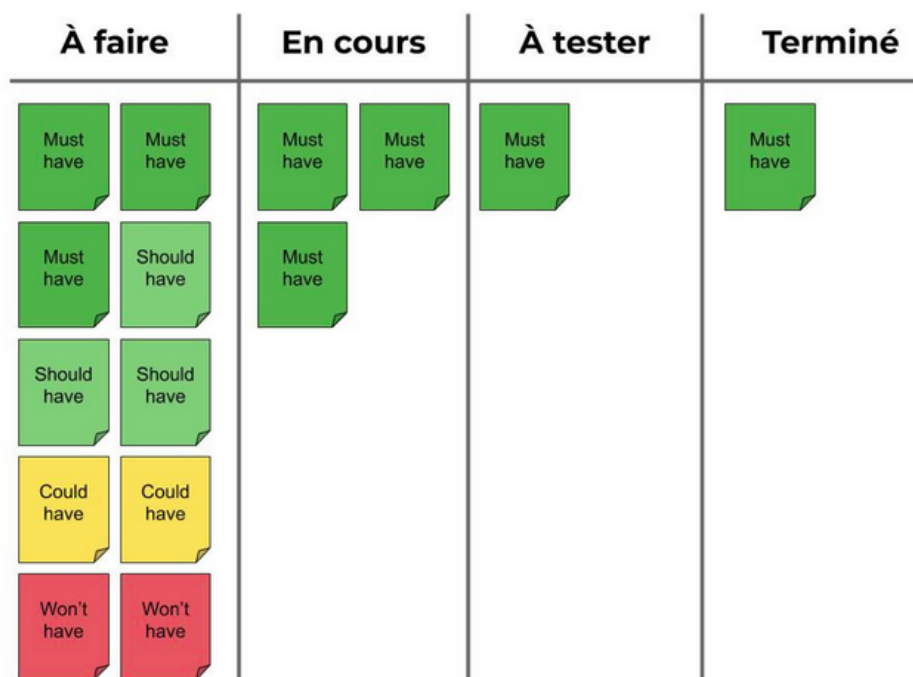


Fig. 1.5 : Méthode Kanban

1.4.3.3 Comparaison entre Scrum et Kanban

Bien que Scrum et Kanban appartiennent tous deux à la famille des approches agiles, ils répondent à des logiques d'organisation différentes. Le tableau 1.1 présente une comparaison synthétique entre ces deux méthodes.

Tab. 1.1 : Comparaison entre Scrum et Kanban

Critère	Scrum	Kanban
Type	Cadre de travail structuré	Méthode de gestion visuelle
Organisation	Sprints fixes	Flux continu
Rôles	Rôles définis	Pas de rôles imposés
Réunions	Cérémonies formelles	Réunions selon le besoin
Adaptation	Ajustements à chaque sprint	Ajustements continus

1.4.4 Méthodologie adoptée

Pour ce projet, une démarche proche de Kanban a été privilégiée. Ce choix s'explique par la nature évolutive du travail réalisé, où plusieurs tâches devaient être traitées progressivement selon leur priorité, leur faisabilité technique et la disponibilité de l'environnement de test.

Le suivi du projet s'est appuyé sur une organisation simple et visuelle, permettant de distinguer les tâches à faire, en cours, terminées ou bloquées. Cette approche a facilité le suivi de l'avancement, tout en laissant la souplesse nécessaire pour réagir rapidement aux contraintes rencontrées, qu'elles soient liées au matériel, aux outils logiciels ou aux scénarios en cours de stabilisation.

Cette manière de travailler s'est révélée particulièrement adaptée à un projet d'automatisation embarquée, où certaines étapes nécessitent plusieurs essais, des phases de debug successives et des ajustements continus.

1.5 Conclusion

Dans ce chapitre, nous avons présenté le cadre général du projet en introduisant l'entreprise STMicroelectronics, son implantation en Tunisie ainsi que le contexte dans lequel s'inscrit ce travail de fin d'études. Nous avons également mis en évidence la problématique liée à la validation manuelle des scénarios de boot sécurisé et défini les objectifs poursuivis à travers le développement du framework d'automatisation.

Enfin, nous avons justifié le choix d'une démarche de travail flexible, adaptée à la nature évolutive du projet. Cette mise en contexte constitue une base importante pour la

suite du rapport. Le chapitre suivant sera consacré à l'analyse des besoins ainsi qu'à la conception de l'architecture de la solution proposée.

Étude théorique

2.1 Introduction

Ce chapitre a pour objectif de présenter les notions théoriques et l’environnement technologique nécessaires à la compréhension du projet réalisé. Dans le cadre de ce travail, il ne s’agit pas seulement de développer un outil logiciel, mais de concevoir une solution d’automatisation capable d’interagir avec des composants matériels, des outils de développement STM32, des mécanismes de sécurité embarquée et des processus de validation structurés.

Dans une première partie, nous présentons les fondements des tests logiciels ainsi que l’intérêt de leur automatisation dans un contexte industriel. Ensuite, nous introduisons les microcontrôleurs STM32, leurs principales caractéristiques et les mécanismes de sécurité qui interviennent dans les scénarios de boot sécurisé étudiés. Enfin, nous détaillons l’environnement matériel et logiciel utilisé, ainsi que la contribution de l’intelligence artificielle dans l’analyse des incidents et l’assistance au diagnostic.

2.2 Fondamentaux des tests logiciels

Les tests logiciels représentent une étape essentielle dans le cycle de développement d’un système informatique. Ils permettent de vérifier qu’un logiciel répond correctement aux exigences attendues, qu’il fonctionne de manière stable et qu’il conserve un comportement fiable dans différentes conditions d’exécution. Dans le domaine des systèmes embarqués, cette dimension est encore plus importante, car les erreurs ne concernent pas uniquement le code, mais également l’interaction entre le logiciel, le matériel et l’environnement d’exécution.

Dans un contexte industriel, les campagnes de validation jouent donc un rôle déterminant dans l’amélioration de la qualité des produits. Elles permettent d’identifier les anomalies, de prévenir les régressions et de fiabiliser les nouvelles fonctionnalités avant leur intégration dans un cycle de développement plus large.

2.2.1 Classification des tests

Les tests logiciels peuvent être classés selon plusieurs critères. La classification la plus courante repose soit sur le niveau de connaissance du système testé, soit sur le niveau auquel le test intervient dans le cycle de développement. Cette distinction permet d'adopter une stratégie de validation adaptée à la nature du besoin.

De manière générale, les approches de test définissent la manière dont le comportement du système est observé, tandis que les niveaux de test indiquent à quelle échelle la validation est réalisée, depuis les unités logicielles jusqu'au système complet.

2.2.1.1 Approches fondamentales des tests logiciels

Les approches de test décrivent la manière dont les scénarios de validation sont conçus. Les plus connues sont les tests boîte noire et les tests boîte blanche. Dans la pratique, ces approches sont souvent complémentaires.

Tests boîte noire

Les tests boîte noire s'intéressent au comportement observable du système sans tenir compte de son implémentation interne. Le testeur fournit des entrées, observe les sorties et vérifie si le comportement obtenu est conforme aux attentes. Cette approche est particulièrement adaptée à la validation fonctionnelle, car elle permet de se concentrer sur les exigences métier sans dépendre de la structure du code.

Dans le cadre d'un système embarqué, ce type de test peut consister, par exemple, à vérifier qu'un message attendu apparaît sur l'UART après une séquence de démarrage, sans avoir à examiner directement le code exécuté.

Boite noire

Visibilité :

- Entrée
- Sortie
- Structure (Code)

Référence :

- Les spécifications

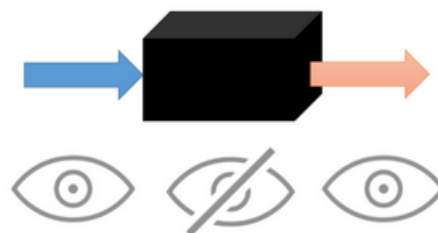


Fig. 2.1 : Tests boîte noire

Tests boîte blanche

Les tests boîte blanche reposent sur une connaissance du code source et de la structure interne du système. Ils permettent d'analyser les chemins d'exécution, les conditions logiques, les boucles et les interactions internes du programme. Cette approche est par-

ticulièrement utile pour les tests unitaires ou pour les phases de développement où l'on cherche à vérifier la qualité interne du code.

Même si le projet présenté dans ce rapport ne consiste pas directement à développer le firmware embarqué lui-même, certains mécanismes d'automatisation s'appuient sur une connaissance détaillée du comportement attendu du système, ce qui rapproche partiellement certaines analyses d'une logique boîte blanche.

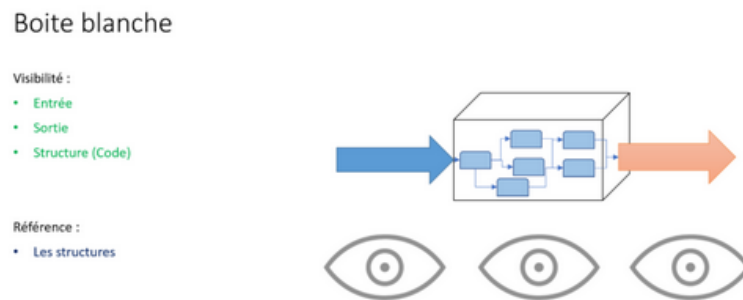


Fig. 2.2 : Tests boîte blanche

Tests boîte grise

Les tests boîte grise représentent une approche intermédiaire. Le testeur ne dispose pas forcément d'un accès complet au code source, mais possède néanmoins une certaine connaissance de l'architecture interne, des interfaces ou des composants du système. Cette approche est fréquente dans les projets de validation embarquée, où l'on connaît le fonctionnement général du système sans pour autant intervenir directement sur toute son implémentation.

Dans notre projet, cette approche est particulièrement représentative, car la validation repose à la fois sur des éléments observables (UART, résultats des outils, états de sécurité) et sur une compréhension technique de la chaîne de boot sécurisée.

2.2.1.2 Hiérarchie des niveaux de test

Les niveaux de test permettent de structurer la validation selon l'échelle du système observé. Chaque niveau répond à un objectif particulier et contribue à la qualité globale du produit final.

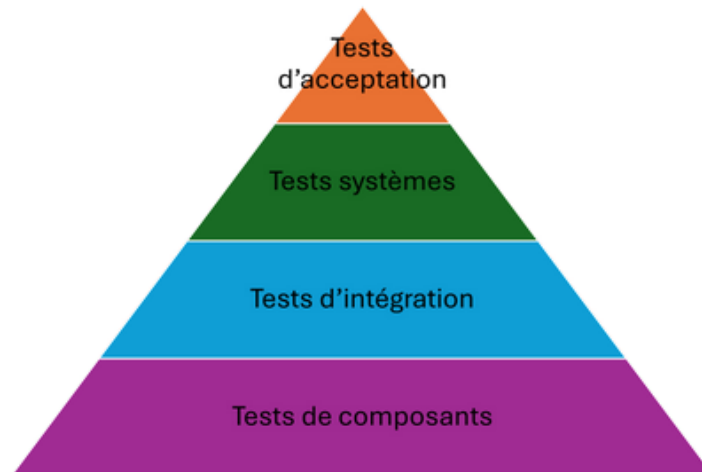


Fig. 2.3 : Hiérarchie des niveaux de test

Tests unitaires

Les tests unitaires constituent le premier niveau de validation. Ils visent à vérifier le comportement d'unités logiques élémentaires telles qu'une fonction, une méthode ou un module isolé. Leur objectif est de détecter rapidement les erreurs de programmation avant que celles-ci ne se propagent à des niveaux plus globaux.

Tests d'intégration

Les tests d'intégration portent sur les interactions entre plusieurs composants. Ils permettent de s'assurer que les modules fonctionnent correctement ensemble et que les échanges de données sont cohérents. Dans les systèmes embarqués, ils concernent souvent l'intégration entre logiciel, outils externes et matériel cible.

Tests système

Les tests système évaluent le comportement de l'ensemble de la solution dans un environnement proche des conditions réelles d'utilisation. Ils permettent de vérifier que les différents sous-ensembles fonctionnent de manière cohérente lorsqu'ils sont assemblés. Les scénarios automatisés de boot sécurisé réalisés dans ce projet s'inscrivent principalement dans cette catégorie.

Tests de validation

Les tests de validation correspondent à la dernière étape du processus. Ils permettent de vérifier que le système développé répond effectivement au besoin métier exprimé. Dans notre cas, il s'agit de s'assurer que les scénarios de boot sécurisé se déroulent correctement sur les cartes cibles, que les preuves de validation sont correctement collectées et que le résultat final est exploitable.

2.2.2 Automatisation des tests

2.2.2.1 Définition

L'automatisation des tests consiste à utiliser des scripts, des outils logiciels ou des frameworks spécialisés afin d'exécuter automatiquement des scénarios de validation. Cette approche permet de réduire les tâches répétitives, d'améliorer la vitesse d'exécution et de rendre les résultats plus homogènes d'une campagne à l'autre.

Dans les environnements techniques complexes, l'automatisation ne se limite pas à déclencher des tests. Elle doit également gérer les dépendances, piloter les outils nécessaires, collecter les preuves d'exécution et organiser les résultats de manière structurée.

2.2.2.2 Tests manuels

Les tests manuels reposent sur l'intervention directe d'un opérateur. Celui-ci prépare les prérequis, lance les outils, exécute les étapes du scénario puis interprète les résultats obtenus. Cette approche peut être pertinente pour les premières explorations, les cas spécifiques ou les vérifications ponctuelles.

Cependant, elle présente plusieurs limites importantes : le temps de réalisation est élevé, la reproductibilité n'est pas toujours garantie, et les risques d'erreur humaine augmentent lorsque les séquences deviennent longues ou complexes.

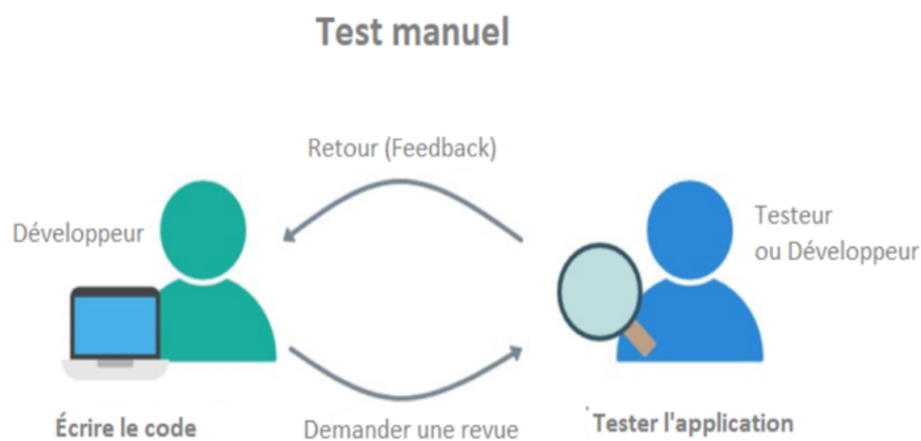


Fig. 2.4 : Processus d'un test manuel

2.2.2.3 Tests automatisés

Les tests automatisés reposent sur l'exécution de scénarios par des scripts ou des frameworks capables de reproduire les étapes de validation de manière systématique. Cette approche améliore la répétabilité, réduit les écarts entre les exécutions et permet d'intégrer plus facilement les campagnes de test dans un processus industriel.

Dans le cadre de ce projet, les tests automatisés s'appuient sur Python, Robot Framework et différents outils STM32 afin de reproduire l'enchaînement réel des opérations nécessaires à la validation du boot sécurisé.

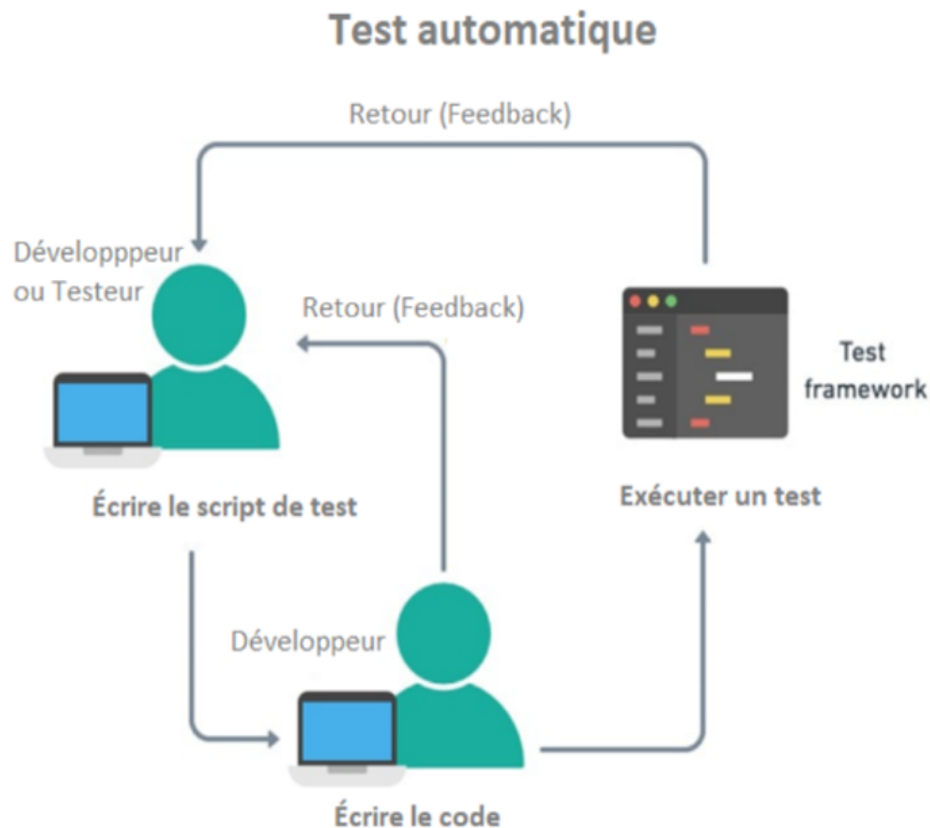


Fig. 2.5 : Processus d'un test automatisé

2.2.2.4 Avantages

L'automatisation présente plusieurs avantages majeurs. Elle permet d'exécuter des scénarios de manière plus rapide et plus fiable, de réduire les manipulations manuelles, d'assurer une meilleure répétabilité et de produire des résultats mieux structurés. Elle simplifie également l'identification des régressions et facilite l'archivage des preuves de validation.

Dans le contexte de ce projet, l'automatisation apporte aussi un bénéfice important en matière de traçabilité. Chaque exécution génère des logs, des artefacts, des preuves UART, des informations de sécurité et des rapports exploitables, ce qui facilite grandement l'analyse en cas d'échec.

2.3 Microcontrôleurs STM32 et sécurité embarquée

2.3.1 Présentation des microcontrôleurs STM32

Un microcontrôleur est un composant électronique programmable qui regroupe, sur une seule puce, un processeur, de la mémoire et différents périphériques d'entrée/sortie. Cette intégration permet de réaliser des fonctions de contrôle, de communication et de traitement dans une grande variété d'applications embarquées.

La famille STM32 de STMicroelectronics est l'une des plus répandues dans le domaine des systèmes embarqués. Elle couvre un très large spectre d'applications grâce à une offre variée de gammes, de performances et de fonctionnalités. Les STM32 sont particulièrement appréciés pour leur fiabilité, la richesse de leur documentation, ainsi que l'écosystème logiciel qui les accompagne.

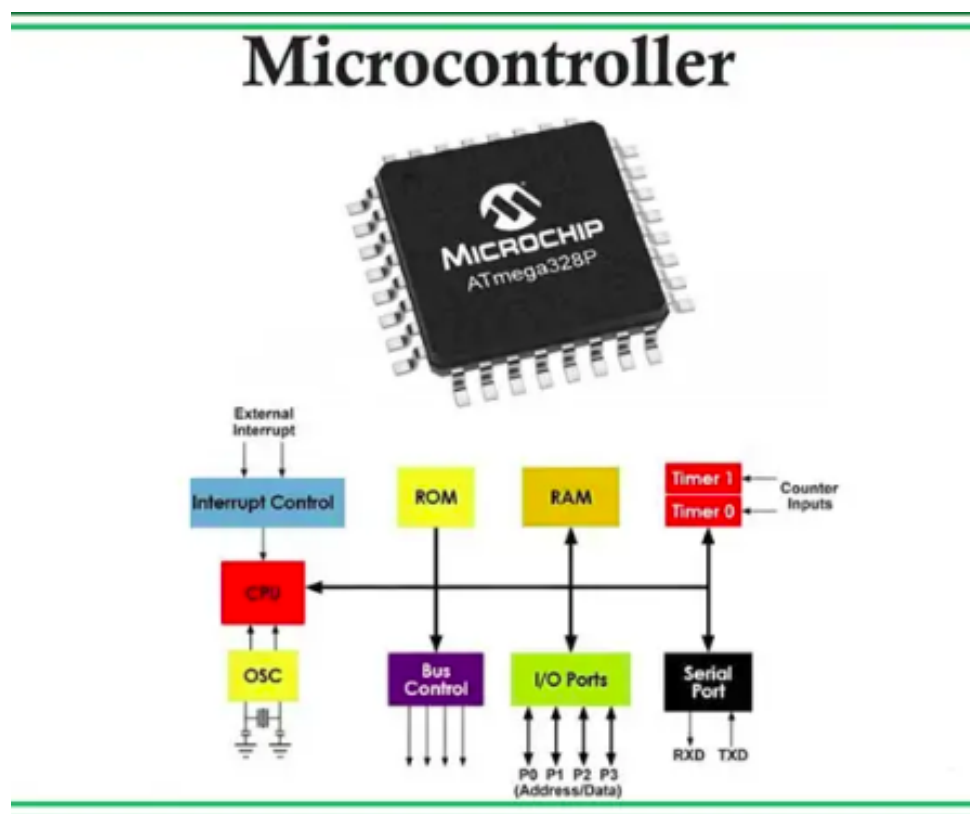


Fig. 2.6 : Composants internes d'un microcontrôleur typique

Dans le cadre de ce projet, l'intérêt des STM32 réside non seulement dans leur architecture matérielle, mais aussi dans les mécanismes de sécurité avancés intégrés dans certaines familles récentes, en particulier celles utilisées dans les scénarios de boot sécurisé étudiés.

2.3.1.1 Architecture ARM Cortex

Les microcontrôleurs STM32 s'appuient sur des cœurs ARM Cortex, principalement de la famille Cortex-M dans le cas des systèmes embarqués classiques. Cette architecture repose sur une logique de type RISC (*Reduced Instruction Set Computer*), qui privilégie un ensemble d'instructions simple et efficace. Ce choix permet d'obtenir un bon compromis entre performances, consommation énergétique et simplicité d'intégration.

La famille Cortex se décline en plusieurs catégories :

- **Cortex-A**, destinée aux systèmes nécessitant un système d'exploitation riche ;
- **Cortex-R**, orientée temps réel critique ;
- **Cortex-M**, conçue pour les systèmes embarqués à ressources limitées.

Dans notre projet, les cartes utilisées reposent principalement sur des microcontrôleurs STM32 intégrant des cœurs adaptés à des applications embarquées sécurisées, ce qui les rend particulièrement pertinents pour l'étude des flows de boot sécurisé.

2.3.1.2 Interfaces multiples

Les STM32 intègrent de nombreuses interfaces de communication. Toutes ne sont pas sollicitées directement dans ce projet, mais certaines jouent un rôle central dans les mécanismes de validation mis en œuvre.

– JTAG et SWD

Les interfaces *JTAG* (*Joint Test Action Group*) et *SWD* (*Serial Wire Debug*) sont utilisées pour établir une communication entre le poste de développement et le microcontrôleur, afin de permettre sa programmation, sa vérification et son débogage.

L'interface JTAG est historiquement l'une des solutions les plus répandues pour le débogage bas niveau. Elle permet notamment la programmation de la mémoire Flash, l'inspection de certains états internes du microcontrôleur et la vérification des connexions matérielles.

L'interface SWD, quant à elle, a été conçue pour les microcontrôleurs ARM Cortex, y compris les STM32. Elle repose sur une liaison série plus légère, utilisant un nombre réduit de broches par rapport à JTAG. Cette caractéristique en fait une solution plus simple à intégrer, tout en restant particulièrement adaptée aux opérations de programmation et de débogage sur les cartes STM32.

Dans le cadre de ce projet, ces interfaces sont exploitées à travers les sondes ST-LINK pour programmer les cartes, déclencher certaines opérations de reset et accéder à différentes informations nécessaires au processus de validation.

– I2C (Inter-Integrated Circuit)

L'I2C est un protocole série synchrone à deux fils, utilisé pour la communication entre composants électroniques sur une même carte. Bien qu'il ne soit pas au cœur du framework développé, il fait partie des interfaces courantes dans l'univers STM32 et illustre la diversité des moyens d'interaction disponibles dans les systèmes embarqués.

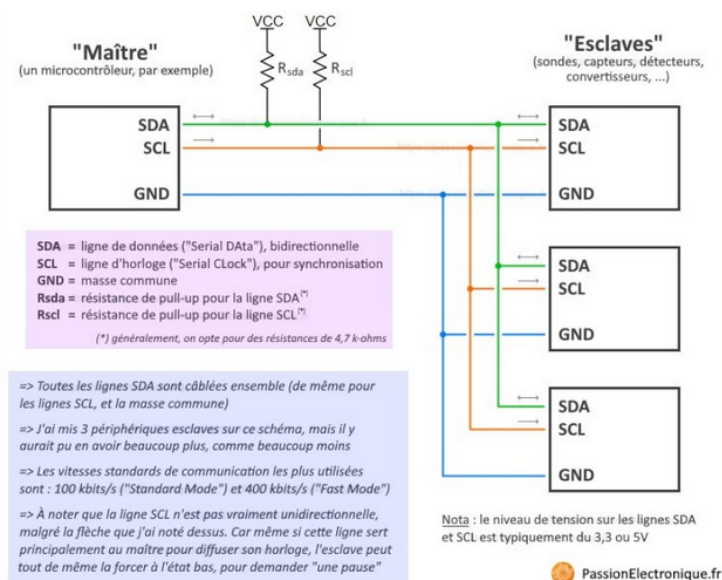


Fig. 2.7 : Exemple de câblage d'une liaison I2C

– UART/USART

L'UART est une interface série asynchrone très largement utilisée dans les systèmes embarqués. Dans ce projet, elle occupe une place centrale, car la sortie UART de la carte cible constitue la preuve principale du bon déroulement du boot. Les messages émis après la phase de démarrage sont capturés puis analysés afin de vérifier la présence des marqueurs attendus.

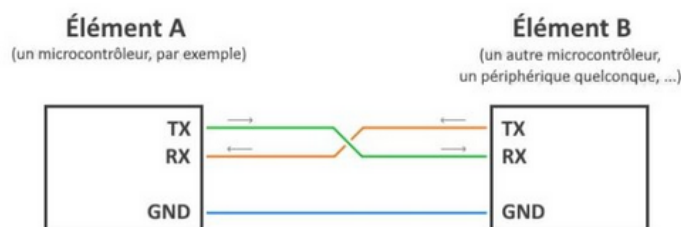


Fig. 2.8 : Exemple de câblage d'une liaison UART

– SPI

Le protocole SPI permet des communications synchrones rapides entre un maître et un ou plusieurs périphériques esclaves. Il est généralement utilisé pour interfacer des

mémoires externes, des écrans ou des capteurs. Même s'il n'est pas directement exploité dans le framework, il fait partie de l'environnement fonctionnel des cartes STM32.

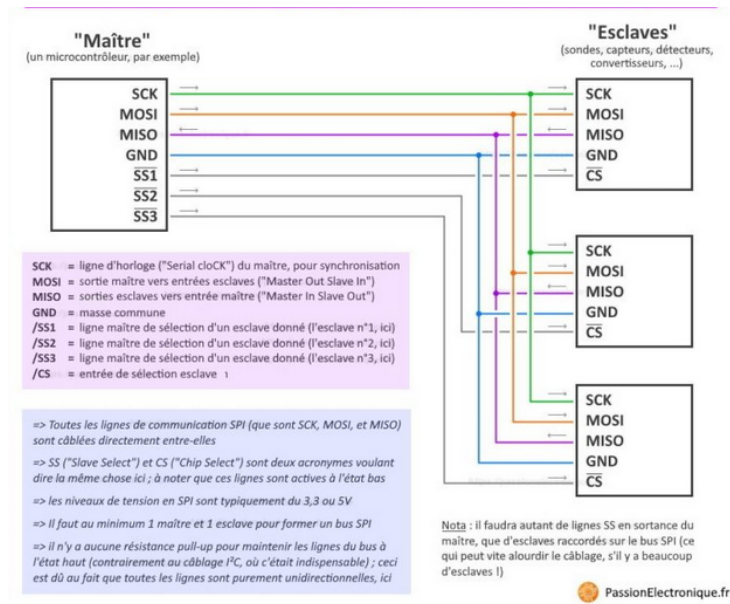


Fig. 2.9 : Exemple de câblage d'une liaison SPI

– USB

L'USB intervient dans le projet comme interface de connexion entre les cartes et le poste de développement. Il permet notamment d'accéder aux interfaces ST-LINK ou aux ports séries virtuels, qui sont indispensables pour la programmation et la collecte des traces.

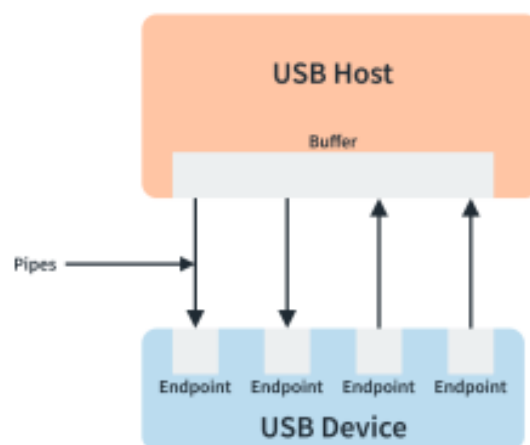


Fig. 2.10 : Exemple de câblage d'une liaison USB

– CAN et Ethernet

Les interfaces CAN et Ethernet sont disponibles sur certaines familles STM32 hautes performances. Elles illustrent l'ouverture des microcontrôleurs STM32 à des environnements industriels et connectés plus larges. Même si elles ne sont pas exploitées directement dans le cadre de notre projet, leur présence témoigne de la polyvalence des plateformes STM32.

2.3.1.3 Mémoire embarquée des STM32

L'organisation mémoire des microcontrôleurs STM32 joue un rôle essentiel dans l'exécution du firmware et dans les mécanismes de sécurité associés au démarrage. On distingue principalement la mémoire Flash, utilisée pour stocker le programme de manière persistante, et la mémoire SRAM, destinée aux données temporaires manipulées pendant l'exécution.

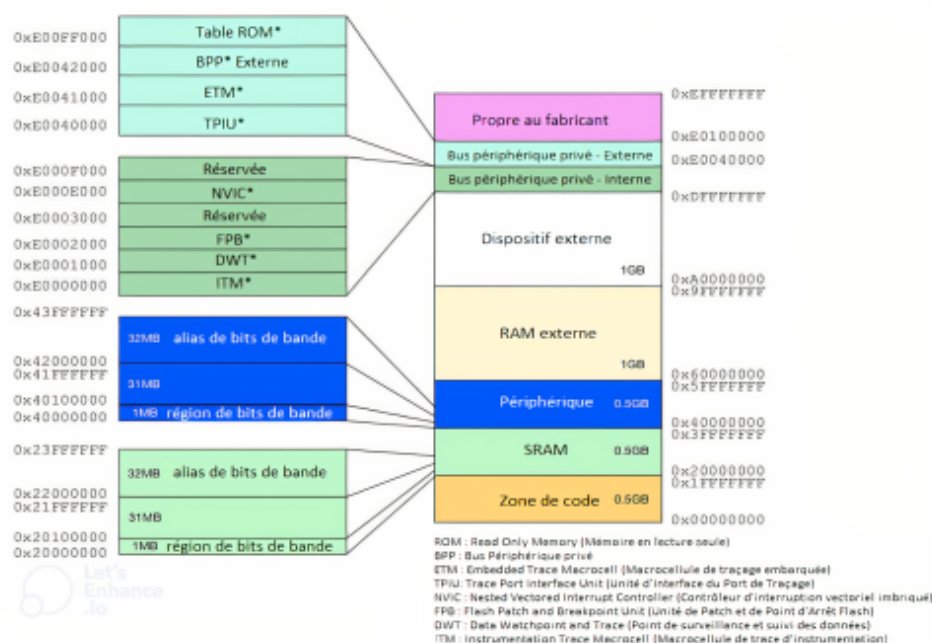


Fig. 2.11 : Organisation mémoire typique d'un microcontrôleur

– Mémoire Flash

La mémoire Flash est une mémoire non volatile utilisée pour stocker le firmware, les images applicatives et certaines données critiques. Dans les STM32, elle est généralement mappée à partir de l'adresse 0x08000000. Selon la configuration de démarrage, elle peut également être rendue visible à l'adresse 0x00000000, ce qui permet l'exécution directe du code après un reset.

Cette mémoire contient non seulement le programme utilisateur, mais aussi différents éléments importants liés au démarrage sécurisé, aux configurations de protection et aux

mécanismes de provisioning.

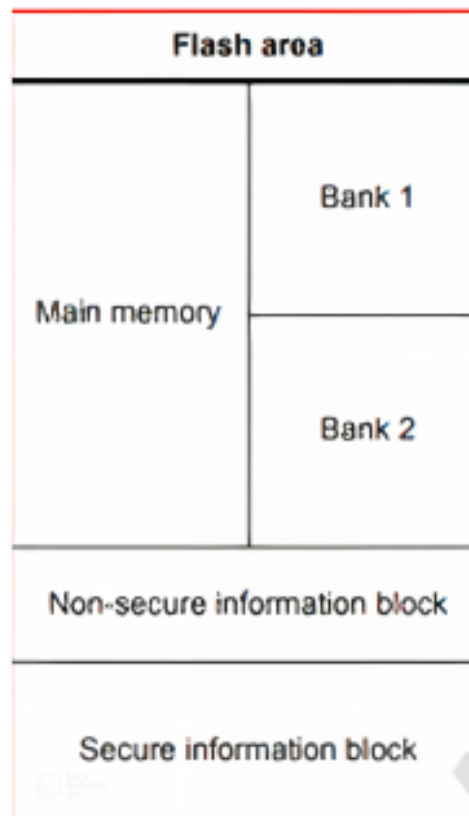


Fig. 2.12 : Organisation simplifiée d'une zone Flash

– SRAM et autres régions mémoire

La SRAM est utilisée pour stocker les variables, la pile, les buffers et les données temporaires nécessaires au fonctionnement du programme. D'autres régions mémoire peuvent également être réservées aux périphériques, aux registres système, aux bootloaders internes ou à certains blocs spécifiques liés à la sécurité.

Dans le cadre de notre projet, l'organisation mémoire est importante, car elle conditionne la manière dont les images Secure et Non-Secure sont placées, générées et exécutées.

2.3.2 Mécanismes de sécurité des STM32

Les microcontrôleurs STM32 récents intègrent plusieurs mécanismes de sécurité permettant de protéger le code, les données sensibles et le cycle de vie du produit. Ces mécanismes sont directement liés aux scénarios validés dans ce projet.

2.3.2.1 Secure Boot

Le *Secure Boot* désigne le processus qui garantit qu'au démarrage, seul un code authentifié et autorisé peut être exécuté. Il repose sur une chaîne de confiance établie dès les premières instructions lancées après le reset.

Dans les STM32 orientés sécurité, cette logique s'appuie sur des mécanismes matériels et logiciels capables de vérifier l'intégrité et l'authenticité des images de firmware. Le but est d'empêcher l'exécution d'un code modifié, corrompu ou non autorisé. Comme le montre la figure 2.13, cette chaîne de démarrage sécurisé commence par une base de confiance initiale, puis se prolonge à travers les différentes étapes de vérification avant d'autoriser l'exécution de l'application finale. La figure met également en évidence les différents modèles de racine de confiance possibles, selon que la sécurité est portée par STMicroelectronics, par l'OEM, ou par une approche hybride combinant plusieurs niveaux de protection.

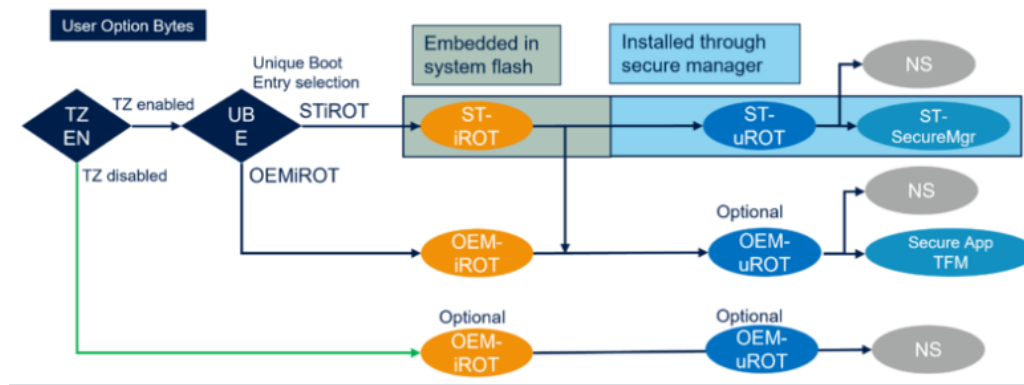


Fig. 2.13 : Chaîne Secure Boot : racine de confiance, vérification du firmware et lancement de l'application

Dans notre projet, la validation de cette séquence ne repose pas seulement sur la réussite théorique des opérations de configuration ou de programmation. Elle s'appuie surtout sur l'observation du comportement réel de la carte cible après le reset, notamment à travers la sortie UART, qui constitue la preuve principale du bon déroulement du boot.

2.3.2.2 TrustZone

TrustZone est une extension de sécurité introduite par ARM afin de séparer matériellement le système en deux mondes distincts : un domaine *Secure* et un domaine *Non-Secure*. Cette séparation ne concerne pas uniquement le processeur, mais aussi certaines zones mémoire, certains périphériques et les échanges entre les différentes parties du système.

Cette architecture permet d'isoler les fonctions critiques, telles que les services de sécurité, les clés ou certaines ressources protégées, du reste de l'application. Comme le montre la figure 2.14, le démarrage du système commence dans un état sécurisé, à partir du projet firmware, avant de transférer l'exécution vers le projet utilisateur non sécurisé. La figure met également en évidence les interactions possibles entre les deux mondes, notamment à travers les appels de fonctions et les mécanismes de rappel (*callbacks*), tout en conservant une séparation logique entre les ressources sensibles et l'application utilisateur.

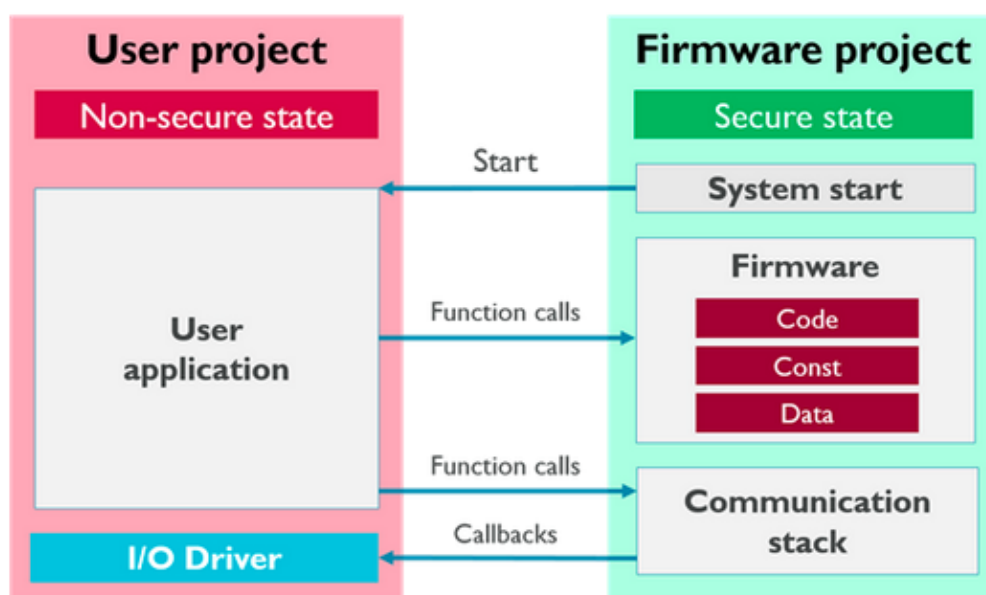


Fig. 2.14 : Séparation fonctionnelle entre les domaines Secure et Non-Secure avec TrustZone

Dans les scénarios de boot sécurisé étudiés dans ce projet, cette séparation joue un rôle fondamental, car elle conditionne l'organisation du firmware, la répartition des responsabilités entre les composants sécurisés et non sécurisés, ainsi que le comportement attendu de la cible au démarrage.

2.3.2.3 Product State

Le *Product State* décrit l'état de sécurité global du microcontrôleur au cours de son cycle de vie. Il permet d'identifier si le composant se trouve encore dans une phase de développement, dans une étape de provisioning, ou dans un état plus sécurisé destiné au déploiement final.

Comme l'illustre la figure 2.15, ce cycle de vie évolue progressivement entre plusieurs états, allant de *Open*, qui correspond à un produit entièrement ouvert, jusqu'aux états *Closed* ou *Locked*, où le niveau de protection devient plus élevé. Des états intermédiaires comme *Provisioning*, *OEMiRoT Provisioned* ou *TZ-Closed* marquent les étapes de sécurisation du composant et influencent directement les droits d'accès à certaines fonctions sensibles.

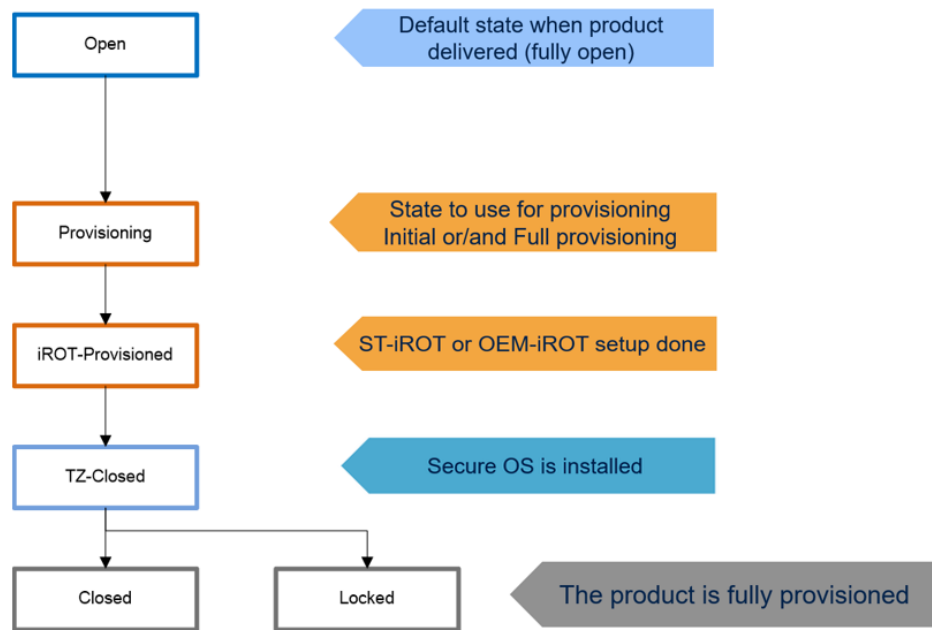


Fig. 2.15 : Cycle de vie du Product State

Dans notre projet, le *Product State* est utilisé comme information de contexte pour enrichir l'analyse des résultats obtenus. Il permet de mieux situer l'état réel du composant au moment du test, sans pour autant constituer, à lui seul, une preuve directe du succès du boot sécurisé.

2.3.2.4 Hide Protection (HDP)

La *Hide Protection* est un mécanisme de protection qui permet de masquer certaines zones sensibles de la mémoire Flash. Son objectif est de limiter l'accès à des données ou à des portions de code jugées critiques, notamment dans les phases sensibles du démarrage sécurisé.

Comme pour les autres informations de sécurité, la valeur de la HDP est exploitée dans ce projet comme une donnée d'analyse complémentaire. Elle peut aider à mieux comprendre certains comportements ou certains échecs, mais elle ne remplace pas la preuve fonctionnelle principale fournie par l'UART.

2.3.3 Scénarios de boot sécurisé STM32

Les scénarios de boot sécurisé étudiés dans ce projet correspondent à différentes organisations de la chaîne de confiance et à différents niveaux de responsabilité entre STMicroelectronics et le fabricant du produit final.

2.3.3.1 ST IROT

Le scénario `ST_IROT` (*ST Immutable Root of Trust*) repose sur une racine de confiance fournie par STMicroelectronics. Dans ce modèle, le point de départ de la chaîne de sécurité est assuré par des mécanismes intégrés et gérés par ST, ce qui fournit une base fiable et robuste pour le démarrage sécurisé.

L'application ou les composants ajoutés par l'utilisateur viennent ensuite s'exécuter au-dessus de cette base. Ce scénario est particulièrement intéressant lorsqu'on souhaite bénéficier d'un niveau de sécurité élevé avec un cadre relativement bien maîtrisé.

2.3.3.2 OEM IROT

Dans le scénario `OEM_IROT`, la racine de confiance est définie par l'équipementier. Ce modèle laisse davantage de liberté dans la conception de la chaîne de sécurité, mais il implique aussi une responsabilité plus importante dans la configuration, la personnalisation et la gestion des mécanismes de protection.

Ce scénario est donc plus flexible, mais également plus exigeant en matière de maîtrise technique.

2.3.3.3 ST IROT / OEMuROT

Le scénario `STIROT_OEMuROT` combine une base sécurisée assurée par ST avec une extension contrôlée par l'OEM. Il s'agit d'une approche hybride qui permet de bénéficier d'une fondation sécurisée robuste tout en conservant une certaine liberté de personnalisation sur les couches supérieures.

Ce modèle est particulièrement adapté aux systèmes nécessitant à la fois des garanties fortes sur la racine de confiance et une adaptation au besoin métier du fabricant final.

2.4 Environnement matériel et logiciel

2.4.1 Environnement matériel

2.4.1.1 Cartes STM32

Cette section présente les principaux équipements matériels utilisés dans le cadre de ce projet, notamment les cartes de développement STM32, les interfaces de programmation et de débogage ST-LINK, ainsi que les composants nécessaires à la validation des scénarios de démarrage sécurisé.

Les cartes de développement STM32 constituent des plateformes essentielles pour la conception, le test et la validation d'applications embarquées basées sur les microcontrôleurs STM32. STMicroelectronics propose plusieurs catégories de cartes répondant à

différents besoins de développement.

Cartes Nucleo

Les cartes Nucleo sont des plateformes de prototypage accessibles et polyvalentes. Elles offrent une compatibilité avec les connecteurs Arduino Uno ainsi qu’avec les connecteurs ST morpho, facilitant l’ajout de cartes d’extension et de modules externes. Elles sont particulièrement adaptées aux phases d’apprentissage, de prototypage rapide et de validation fonctionnelle.

Cartes Discovery

Les cartes Discovery sont destinées à l’évaluation de fonctionnalités spécifiques des microcontrôleurs STM32. Elles intègrent généralement plusieurs périphériques embarqués tels que des capteurs, des écrans, des interfaces audio ou des modules de communication, permettant le développement et la démonstration d’applications plus avancées.

Cartes d’évaluation

Les cartes d’évaluation, ou *Evaluation Boards* (EV), représentent la gamme la plus complète proposée par STMicroelectronics. Elles sont conçues pour offrir un accès étendu aux ressources matérielles du microcontrôleur et intègrent de nombreux périphériques, interfaces de communication et connecteurs d’extension. Ces cartes sont particulièrement adaptées aux études approfondies, à l’évaluation des performances et au développement d’applications industrielles complexes.

La figure 2.16 illustre les principales catégories de cartes STM32 proposées par STMicroelectronics.

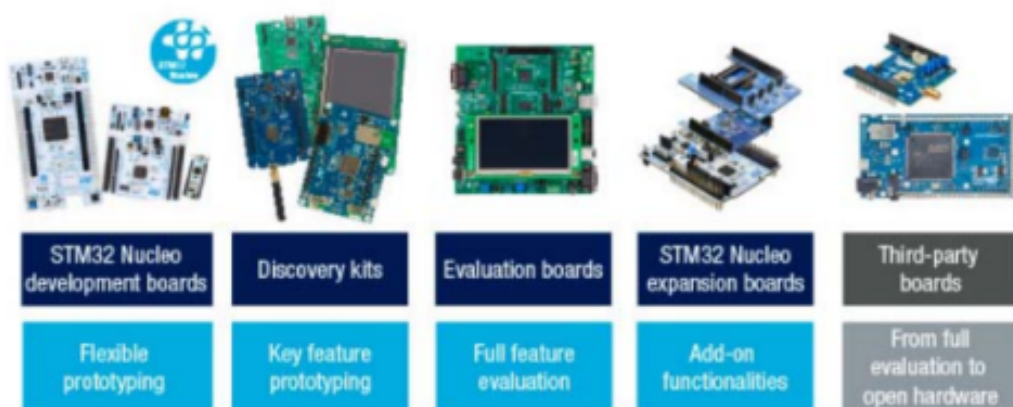


Fig. 2.16 : Différentes familles de cartes STM32

2.4.1.2 ST-LINK

Le ST-LINK est l’outil de programmation et de débogage développé par STMicroelectronics pour les microcontrôleurs STM32. Il constitue un élément essentiel du processus de développement, de test et de validation des applications embarquées. Compatible avec

les interfaces SWD (*Serial Wire Debug*) et JTAG, il permet d'établir une communication directe entre l'ordinateur et le microcontrôleur cible.

Grâce à cette interface, il est possible d'effectuer plusieurs opérations telles que le téléchargement du firmware, l'exécution du débogage pas à pas, la lecture et l'écriture des registres ainsi que l'analyse du comportement du système en temps réel. Comme illustré dans la figure 2.17, le ST-LINK peut être intégré directement sur certaines cartes de développement STM32 ou utilisé sous forme de sonde externe pour programmer et déboguer des cartes personnalisées.

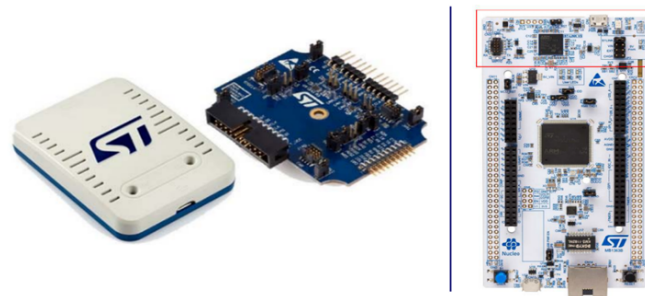


Fig. 2.17 : ST-LINK

En mode programmation, le ST-LINK assure la conversion des commandes USB provenant du poste de développement vers les protocoles SWD ou JTAG utilisés par le microcontrôleur. Cette fonctionnalité permet l'utilisation d'outils tels que STM32CubeIDE ou STM32CubeProgrammer pour programmer, configurer et analyser les microcontrôleurs STM32 de manière efficace. La figure 2.18 présente le principe de connexion entre le ST-LINK et la carte cible.

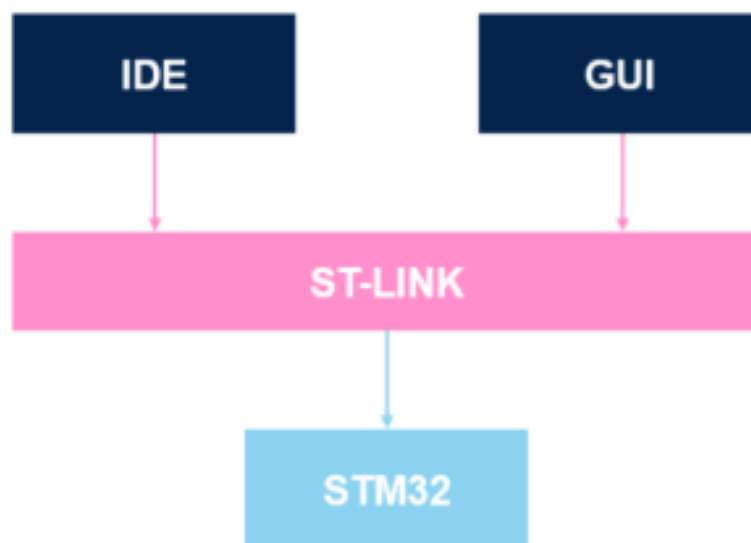


Fig. 2.18 : Connexion ST-LINK

2.4.1.3 UART

Principe

L'UART est une interface série asynchrone très répandue dans les systèmes embarqués. Elle permet l'échange de données entre la carte et un équipement externe via des lignes de transmission et de réception, généralement notées TX et RX.

Utilisation dans la validation du boot

Dans ce projet, l'UART constitue la preuve principale du bon déroulement du boot. Après le provisioning et le reset de la carte cible, la trace UART est capturée pendant une fenêtre de temps déterminée. Les messages obtenus sont ensuite analysés pour identifier les marqueurs attendus selon le scénario exécuté. Cette démarche permet de valider le comportement réel du système au démarrage, ce qui en fait un indicateur métier particulièrement fiable.

2.4.1.4 Bootloader

Le bootloader des microcontrôleurs STM32 est un programme intégré dans la mémoire système qui permet de programmer ou de mettre à jour le firmware sans utiliser obligatoirement un débogueur externe. Il joue un rôle important dans les phases de développement, de maintenance et de récupération du système.

Le mode de démarrage dépend généralement de la configuration des broches `BOOT0` et `BOOT1`. Selon leur état, le microcontrôleur peut démarrer depuis la mémoire Flash principale, depuis la mémoire système contenant le bootloader interne, ou depuis la SRAM.

Les principaux modes de démarrage sont résumés dans le tableau 2.1.

Tab. 2.1 : Modes de démarrage selon l'état des broches de boot

BOOT0	BOOT1	Mode de démarrage
0	X	Mémoire Flash principale
1	0	Mémoire système
1	1	SRAM intégrée

Ce bootloader peut utiliser plusieurs interfaces de communication comme l'UART, l'USB, le SPI, le CAN ou l'I²C, selon la famille STM32 utilisée. Il représente ainsi une solution pratique pour la programmation, la mise à jour du firmware et certaines opérations de maintenance.

2.4.2 Environnement logiciel

2.4.2.1 STM32CubeMX

STM32CubeMX est un outil de configuration utilisé pour préparer les projets STM32 avant la phase de développement. Il permet de configurer les périphériques, les paramètres d'horloge, certains middlewares ainsi que l'organisation générale du projet selon la carte ciblée.

Dans les projets orientés sécurité, cet outil intervient également dans la configuration de paramètres liés à TrustZone et à la séparation entre les domaines *Secure* et *Non-Secure*. Il joue ainsi un rôle important dans la préparation des scénarios de boot sécurisé, en particulier lorsque la structure du projet doit être définie avant la compilation.

Comme le montre la figure 2.19, STM32CubeMX propose une interface graphique centralisant les principales étapes de configuration du projet. Dans le cadre de notre travail, il est surtout utilisé dans les scénarios standards, où le projet doit être généré ou préparé avant son ouverture dans STM32CubeIDE. En revanche, dans les scénarios template, cette étape peut être évitée lorsque le projet existe déjà dans l'environnement STM32CubeFW.

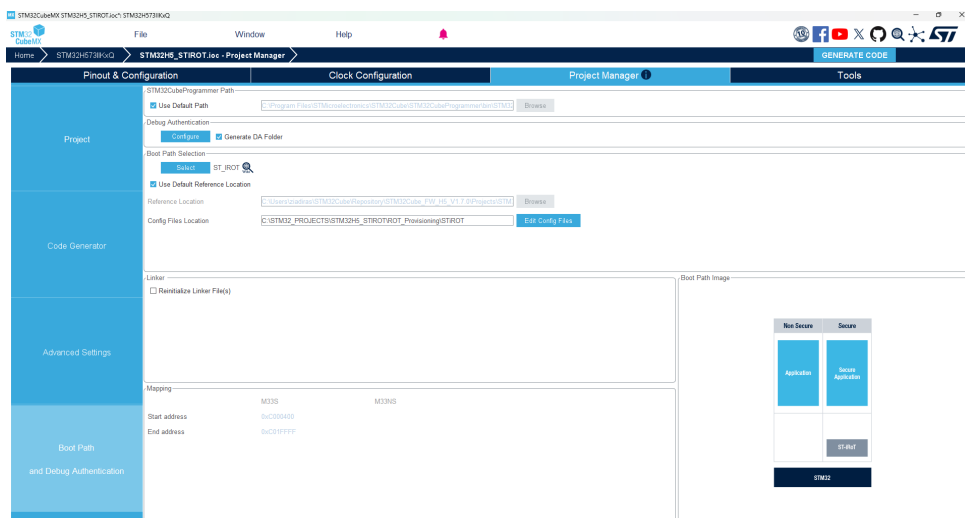


Fig. 2.19 : Interface de STM32CubeMX

2.4.2.2 STM32CubeIDE

STM32CubeIDE est l'environnement de développement intégré utilisé pour manipuler les projets STM32. Il regroupe dans une même interface les fonctions d'édition, de gestion de projet, de compilation et de débogage, ce qui en fait un outil central dans la phase de développement et de validation des applications embarquées.

Dans le cadre de notre projet, STM32CubeIDE est principalement utilisé pour compiler les différentes parties des projets sécurisés, notamment les portions *Secure* et *Non-Secure* lorsque le scénario l'exige. Cette étape est importante, car elle permet de produire

les binaires qui seront ensuite utilisés dans les phases de provisioning, de programmation et de validation du boot.

Comme le montre la figure 2.20, l'outil offre une interface structurée autour de l'explorateur de projet, de l'éditeur de code et des panneaux d'analyse et de build. Même si le débogage interactif n'est pas au cœur du framework développé, les fonctions de debug proposées par STM32CubeIDE restent utiles lors des phases de mise au point ou lorsqu'un comportement inattendu doit être investigué.

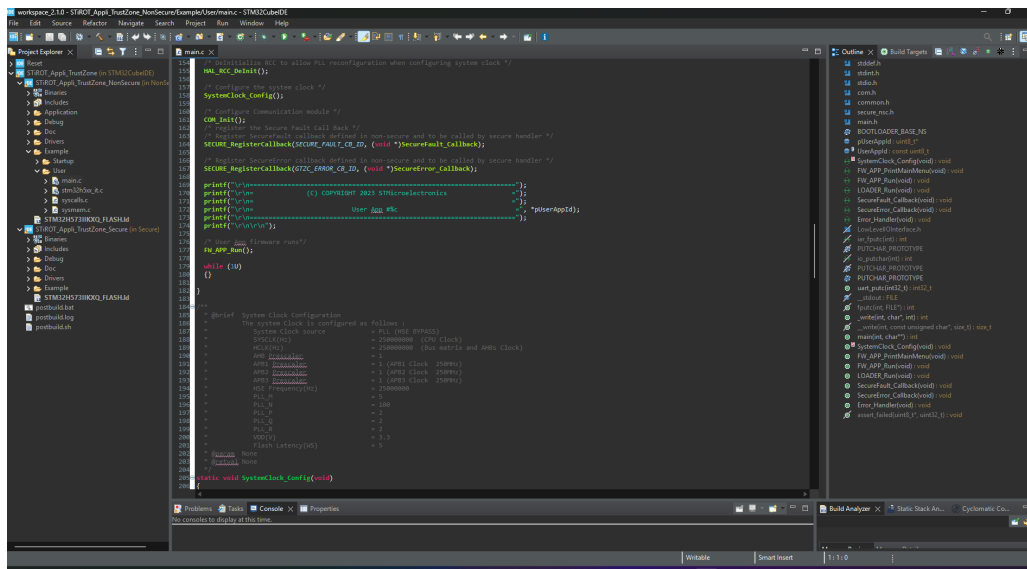


Fig. 2.20 : Interface de STM32CubeIDE

2.4.2.3 STM32CubeProgrammer

Interface GUI

STM32CubeProgrammer dispose d'une interface graphique permettant de programmer les cartes, d'observer certaines informations mémoire, de lire les options de sécurité et d'exécuter différentes opérations système. Comme le montre la figure 2.21, cette interface regroupe les fonctions principales dans une vue visuelle claire, avec les paramètres de connexion, l'accès à la mémoire et les journaux d'exécution. Elle peut être utilisée directement par l'opérateur, mais elle intervient aussi indirectement dans certains scénarios automatisés.

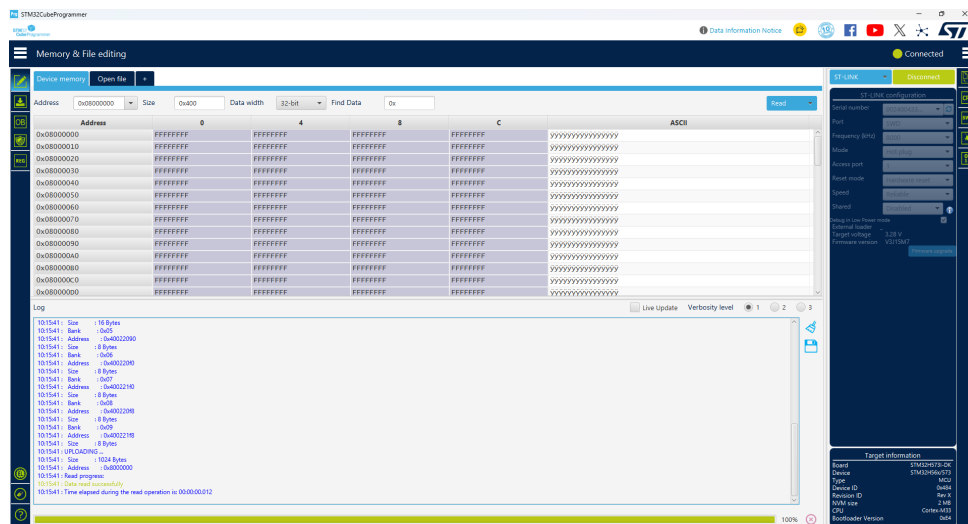


Fig. 2.21 : Interface graphique de STM32CubeProgrammer

Interface CLI

La version en ligne de commande, `STM32_Programmer_CLI`, joue un rôle central dans l'automatisation. Elle permet d'exécuter des opérations de programmation, de lecture d'informations de sécurité, de connexion *hotplug* ou *under reset*, ainsi que différentes commandes nécessaires à l'orchestration du framework. Comme l'illustre la figure 2.22, cette interface fournit un retour textuel détaillé sur la cible et facilite l'intégration de l'outil dans les scripts automatisés.

```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26100.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin>.\STM32_Programmer_CLI.exe -C port=SWD

-----
STM32CubeProgrammer v2.23.0-B08
-----

ST-LINK SN : 002400433232510239353236
ST-LINK FW : V3J15M7
Board : STM32H573I-DK
Voltage : 3.28V
SWD freq : 8000 KHz
Connect mode: Normal
Reset mode : Software reset
Device ID : 0x484
Revision ID : Rev X
Device name : STM32H56x/573
NVM size : 2 MBytes
Device type : MCU
Device CPU : Cortex-M33
BL Version : 0xE4
SFSP Version: v2.5.0
Debug in Low Power mode enabled

C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin>
  
```

Fig. 2.22 : Interface en ligne de commande de STM32CubeProgrammer

2.4.2.4 Trusted Package Creator (TPC)

STM32TrustedPackageCreator est un outil utilisé pour préparer certaines images sécurisées dans les scénarios de boot sécurisé. Il intervient principalement dans la génération ou le packaging des éléments qui seront ensuite provisionnés ou programmés sur la cible, en respectant les contraintes de sécurité imposées par le flow considéré.

Dans le cadre de notre projet, cet outil est utilisé dans les scénarios nécessitant la préparation d'images signées ou structurées avant leur déploiement. Comme le montre la figure 2.23, l'interface de STM32TrustedPackageCreator permet de définir les fichiers d'entrée, les fichiers de sortie ainsi que différents paramètres associés à la génération de l'image sécurisée. Son utilisation s'intègre donc naturellement dans les étapes de préparation précédant le provisioning et la validation du boot.

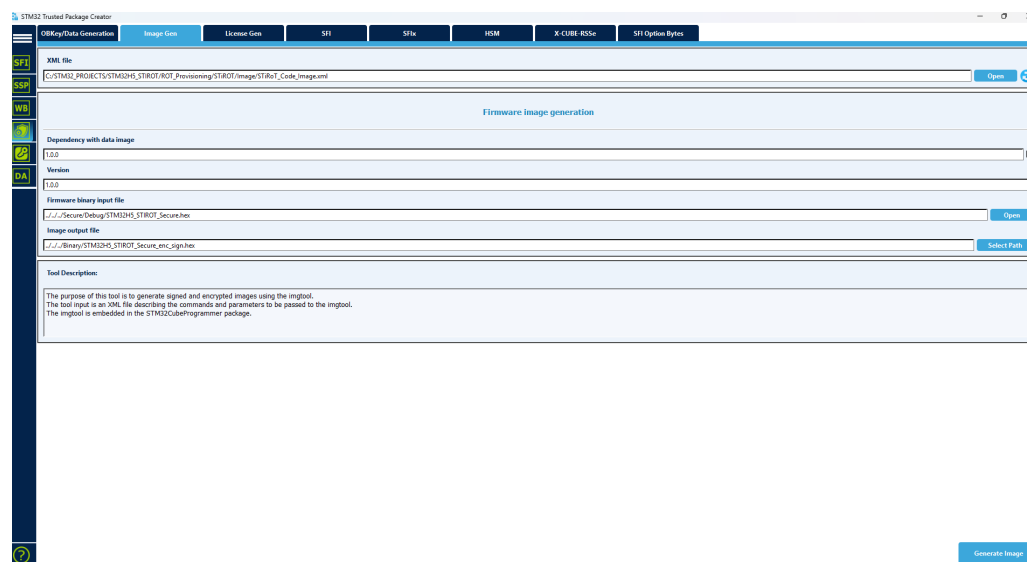


Fig. 2.23 : Interface de STM32TrustedPackageCreator

2.4.2.5 Robot Framework

Robot Framework est un framework d'automatisation open source orienté tests et validation. Il repose sur une syntaxe lisible basée sur des mots-clés, ce qui facilite la structuration des scénarios et l'enchaînement des étapes de test.

Robot Framework constitue dans notre projet le moteur principal d'orchestration des scénarios. Son architecture repose sur des suites de tests, des cas de test, des mots-clés réutilisables, des variables et des bibliothèques externes, notamment en Python. Cette organisation le rend particulièrement adapté à l'automatisation de séquences complexes tout en conservant une bonne lisibilité.

Dans le cadre de notre travail, il a été retenu pour piloter les différents scénarios de boot sécurisé, encapsuler les étapes métier et produire des rapports d'exécution standardisés exploitables après chaque campagne de validation.



Fig. 2.24 : Robot Framework

2.4.2.6 Python : Langage de programmation et environnement de développement intégré (IDE) PyCharm

Python

Python est un langage de programmation largement utilisé dans le domaine de l'automatisation grâce à sa syntaxe simple, sa lisibilité et la richesse de son écosystème logiciel. Dans le cadre de ce projet, Python a joué un rôle central dans le développement du framework d'automatisation des scénarios de validation Secure Boot sur les microcontrôleurs STM32.

Ses nombreuses bibliothèques permettent d'interagir avec les outils STM32, de traiter les fichiers générés, de capturer et analyser les logs, ainsi que de piloter des applications externes. Des bibliothèques telles que PyAutoGUI sont notamment utilisées pour automatiser les interfaces graphiques lorsque certaines opérations ne peuvent pas être réalisées directement via des interfaces en ligne de commande.

Grâce à ses capacités de scripting, Python facilite également l'orchestration des différentes étapes du processus de validation, telles que la génération de projets, la compilation, le provisioning, la collecte des preuves UART et la génération des rapports. Cette flexibilité en fait un choix particulièrement adapté aux projets d'automatisation de tests embarqués.

PyCharm comme IDE pour Python

PyCharm est un environnement de développement intégré (IDE) développé par JetBrains et spécialement conçu pour le développement d'applications Python. Il fournit un ensemble d'outils permettant d'améliorer la productivité et la qualité du code.

Parmi ses principales fonctionnalités figurent la coloration syntaxique avancée, l'autocomplétion intelligente, la détection automatique des erreurs, le débogage intégré ainsi que la gestion des environnements virtuels. Ces outils facilitent le développement et la maintenance des scripts Python utilisés dans le framework d'automatisation.

PyCharm offre également une intégration native avec les systèmes de gestion de ver-

sions tels que Git, permettant un suivi efficace des modifications du code source et une meilleure collaboration entre les membres de l'équipe. Grâce à ces fonctionnalités, il constitue un environnement adapté au développement de solutions d'automatisation complexes comme celles mises en œuvre dans ce projet.

2.4.2.7 Git : système de gestion de versions

Git est un système de gestion de versions distribué largement utilisé dans les projets de développement logiciel. Il permet de suivre l'évolution du code source, de conserver un historique précis des modifications et de faciliter le travail collaboratif au sein d'une équipe.

Comme le montre la figure 2.25, Git repose sur une architecture décentralisée dans laquelle chaque développeur dispose d'une copie locale complète du dépôt. Le cycle de travail s'appuie sur des opérations comme le *clone*, le *add*, le *commit*, le *pull* et le *push*, qui permettent de gérer localement les changements avant leur synchronisation avec le dépôt distant. Cette organisation offre une grande souplesse pour développer de nouvelles fonctionnalités, corriger des anomalies ou tester des modifications sans affecter directement la version principale du projet.

Dans le cadre de notre projet, Git a été utilisé pour assurer le suivi et la maintenance de l'ensemble des composants du framework, notamment les scripts Python, les suites Robot Framework, les fichiers de configuration, les modules de diagnostic et les outils de génération de rapports. Son utilisation a contribué à structurer l'évolution du projet et à intégrer plus facilement les différentes améliorations apportées au framework.

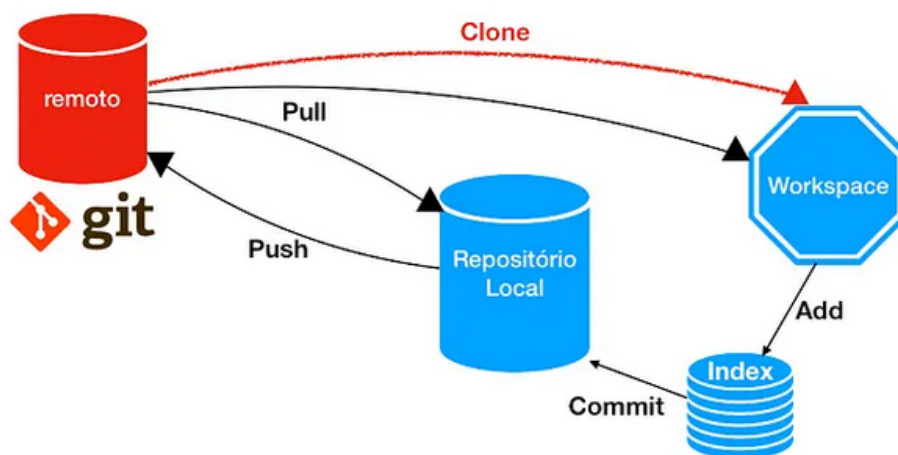


Fig. 2.25 : Architecture d'un système de gestion de versions distribué Git

2.4.2.8 Tkinter

Tkinter est la bibliothèque graphique standard de Python. Elle permet de concevoir des interfaces simples à travers des fenêtres, des boutons, des zones de texte et différents composants de saisie.

Dans le cadre de ce projet, Tkinter est utilisé pour développer l’interface de lancement du framework. Cette interface permet à l’utilisateur de sélectionner le scénario, la carte cible, certains paramètres d’exécution ainsi que quelques options de contrôle, sans recourir systématiquement à la ligne de commande.

2.4.2.9 SikuliX

SikuliX est un outil d’automatisation fondé sur la reconnaissance visuelle des éléments affichés à l’écran. Il permet d’interagir avec des applications graphiques lorsqu’aucun pilotage direct via API ou ligne de commande n’est disponible.

Dans notre projet, SikuliX est utilisé pour automatiser certaines interactions avec les outils GUI de l’écosystème STM32. Même si cette approche reste plus sensible à l’environnement d’exécution, elle demeure utile pour traiter certaines opérations difficiles à automatiser autrement.

2.5 Intelligence artificielle appliquée à l’automatisation des tests

2.5.1 Intelligence artificielle et modèles LLM

2.5.1.1 Concepts généraux

L’intelligence artificielle regroupe un ensemble de méthodes permettant à un système de traiter des informations, d’identifier des régularités et d’assister certaines tâches d’analyse ou de décision. Dans le domaine logiciel, elle est de plus en plus utilisée pour exploiter des volumes importants de données, interpréter des logs et accélérer le diagnostic des anomalies.

2.5.1.2 Fonctionnement des LLM

Les modèles de langage de grande taille, ou LLM (*Large Language Models*), sont capables d’analyser du texte, d’en extraire les éléments importants et de générer une réponse structurée. Ils ne remplacent pas une logique métier déterministe, mais ils peuvent enrichir l’analyse, reformuler des résultats techniques et faciliter l’interprétation d’incidents complexes.

2.5.2 Diagnostic assisté par IA

2.5.2.1 Analyse des logs

Dans les environnements de validation, les logs produits après une exécution sont souvent nombreux et hétérogènes. Leur analyse manuelle peut devenir longue, surtout lorsqu'il faut croiser plusieurs sources d'information. L'IA permet d'exploiter ces données de manière plus synthétique en mettant en évidence les éléments significatifs et les indices utiles au diagnostic.

2.5.2.2 Classification des erreurs

L'analyse assistée par IA peut également aider à regrouper certaines anomalies par catégories et à rapprocher des incidents présentant des caractéristiques similaires. Cette structuration facilite la lecture des échecs et permet d'orienter plus rapidement l'investigation.

2.5.2.3 Génération de recommandations

Au-delà de l'identification des anomalies, l'IA peut formuler des recommandations ou proposer des pistes d'analyse. Cela permet de gagner du temps lors du débogage et d'obtenir une première interprétation exploitable des résultats observés.

2.5.3 Apport de l'IA dans le projet

2.5.3.1 Analyse automatique des incidents

Dans le framework développé, l'IA intervient après l'exécution du scénario. Une fois les résultats collectés, un incident structuré est généré à partir des logs Robot Framework, de la trace UART, des sorties de sous-processus et des informations de sécurité récupérées via STM32CubeProgrammer.

2.5.3.2 Aide au débogage

À partir de cet incident, une première logique locale permet de classer certaines erreurs et d'estimer l'état probable du boot. Ensuite, un modèle de langage enrichit cette analyse en produisant une synthèse plus lisible et des éléments d'aide au diagnostic. Cette approche ne remplace pas l'expertise de l'ingénieur, mais elle améliore la rapidité d'exploitation des résultats.

2.5.3.3 Génération de rapports

L’IA contribue également à améliorer la qualité du reporting final. Au lieu de se limiter à un simple statut de succès ou d’échec, le framework peut produire une analyse textuelle plus claire, mettant en évidence les causes probables, les éléments de contexte et les recommandations utiles à l’investigation.

2.6 Conclusion

Dans ce chapitre, nous avons présenté les principaux éléments théoriques et technologiques nécessaires à la compréhension du projet. Nous avons d’abord rappelé les fondements des tests logiciels et l’intérêt de leur automatisation dans un contexte industriel. Nous avons ensuite introduit les microcontrôleurs STM32, leurs principales interfaces, ainsi que les mécanismes de sécurité mobilisés dans les scénarios de boot sécurisé étudiés.

Nous avons également décrit l’environnement matériel et logiciel utilisé pour mettre en œuvre la solution, avant de présenter le rôle de l’intelligence artificielle dans l’analyse des incidents et l’aide au diagnostic. L’ensemble de ces éléments constitue la base nécessaire pour aborder, dans le chapitre suivant, la réalisation concrète du framework d’automatisation développé dans ce projet.

Réalisation et évaluation de la solution

3.1 Introduction

Ce chapitre est consacré à la mise en œuvre concrète de la solution développée dans le cadre de ce projet. Après avoir présenté le contexte général ainsi que les notions théoriques nécessaires à la compréhension du sujet, il convient à présent de décrire la manière dont le framework a été conçu, structuré et exploité pour automatiser la validation des scénarios de boot sécurisé sur cartes STM32.

L’objectif poursuivi ne se limitait pas à automatiser une suite d’actions répétitives. Il s’agissait plutôt de mettre en place une véritable chaîne de validation, capable d’orchestrer les outils STM32, de piloter les scénarios de test, de capturer les preuves d’exécution, de consolider les incidents et de restituer les résultats sous une forme exploitable. Une telle approche répond à un besoin réel d’industrialisation des campagnes de validation, dans un contexte où la répétabilité, la traçabilité et la qualité du diagnostic sont devenues aussi importantes que l’exécution elle-même.

Dans ce chapitre, nous présentons d’abord la configuration matérielle utilisée dans le banc de validation. Nous détaillons ensuite le processus général des tests automatiques ainsi que l’architecture globale du framework. Enfin, nous décrivons la mise en œuvre de la solution, depuis le lancement des scénarios jusqu’au post-traitement des résultats, avant de discuter les résultats obtenus.

3.2 Configuration matérielle

La réalisation de ce projet repose sur une configuration matérielle spécifique, mise en place pour permettre l’exécution fiable des scénarios de boot sécurisé. Contrairement à une simple automatisation logicielle, le framework développé interagit directement avec des cartes STM32, des interfaces de programmation et de communication, ainsi qu’avec une carte de supervision dédiée au déclenchement du reset matériel. Cette configuration

constitue ainsi le socle physique indispensable à la validation.

3.2.1 Cartes cibles utilisées

Les campagnes de validation ont été menées principalement sur deux cartes cibles : la STM32H573I-DK et la STM32H7S78-DK. Ces deux plateformes ont été retenues car elles permettent de couvrir des scénarios de sécurité récents tout en offrant un environnement réaliste pour l'évaluation des mécanismes de boot sécurisé. Leur utilisation a permis de vérifier que le framework développé reste applicable à plusieurs familles STM32 et ne se limite pas à une seule cible.

La carte STM32H573I-DK a été utilisée dans plusieurs scénarios de validation liés au secure boot, en particulier pour les flows reposant sur les mécanismes de sécurité de la famille H5. La carte STM32H7S78-DK, quant à elle, a permis d'élargir le périmètre de test et de renforcer la représentativité du framework sur des plateformes différentes.

Les cartes cibles utilisées dans le projet sont présentées dans la figure 3.1, qui regroupe les deux principales plateformes matérielles mobilisées pour les campagnes de validation.



(a) STM32H573I-DK



(b) STM32H7S78-DK

Fig. 3.1 : Cartes cibles STM32 utilisées dans le cadre du projet

3.2.2 Carte supervisor

En plus des cartes cibles, le projet s'appuie sur une troisième carte, la STM32U599J-DK, utilisée comme carte *supervisor*. Son rôle n'est pas de servir de cible de validation au même titre que les cartes H5 ou H7, mais d'assurer une fonction de supervision matérielle dans la séquence de test. Elle est programmée avec un firmware spécifique, `supervisor.elf`, dont l'objectif est de générer un reset matériel fiable sur la carte cible.

Ce choix a été motivé par la nécessité de rendre la séquence de redémarrage plus robuste. Les approches antérieures reposaient sur des méthodes plus fragiles, tandis que l'introduction de cette carte supervisor a permis de mieux contrôler le moment du reset,

ce qui est particulièrement important lorsque la capture UART doit être synchronisée avec le redémarrage effectif de la cible.

La carte supervisor utilisée dans le banc de validation est présentée dans la figure 3.2.

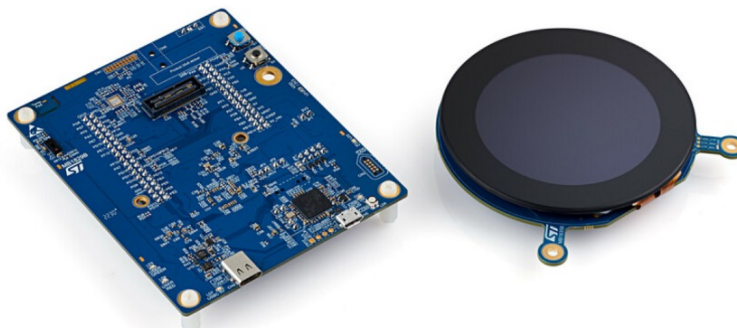


Fig. 3.2 : Carte supervisor STM32U599J-DK

3.2.3 Interfaces de connexion et câblage

Le bon fonctionnement du framework repose également sur l'ensemble des interfaces matérielles mises en place entre le poste de test, la carte cible et la carte supervisor. Trois éléments jouent ici un rôle essentiel : l'interface ST-LINK, la liaison UART et le câblage de reset entre la carte de supervision et la cible.

Les interfaces ST-LINK assurent la programmation et l'accès aux cartes STM32 depuis le poste de développement. Elles sont utilisées pour les opérations de flash, de reset et de lecture de certaines informations de sécurité. L'UART, quant à elle, constitue le principal canal de preuve métier, puisque la validation du boot repose sur l'analyse des messages émis par la carte cible après le redémarrage. Enfin, la carte supervisor est reliée à la cible par une liaison dédiée au reset matériel, permettant de piloter précisément le redémarrage de la carte testée.

Comme le montre la figure 3.3, le poste de développement communique avec la carte cible STM32H573 à travers une liaison USART-VCP pour la récupération des traces UART, ainsi qu'une liaison USB-STLINK pour les opérations de programmation. En parallèle, une seconde liaison USB-STLINK relie le poste à la carte STM32U599 utilisée comme supervisor. Cette dernière est connectée à la cible via une ligne de reset vers la broche **NRST**, ce qui permet de déclencher un reset matériel contrôlé au cours du scénario de validation.

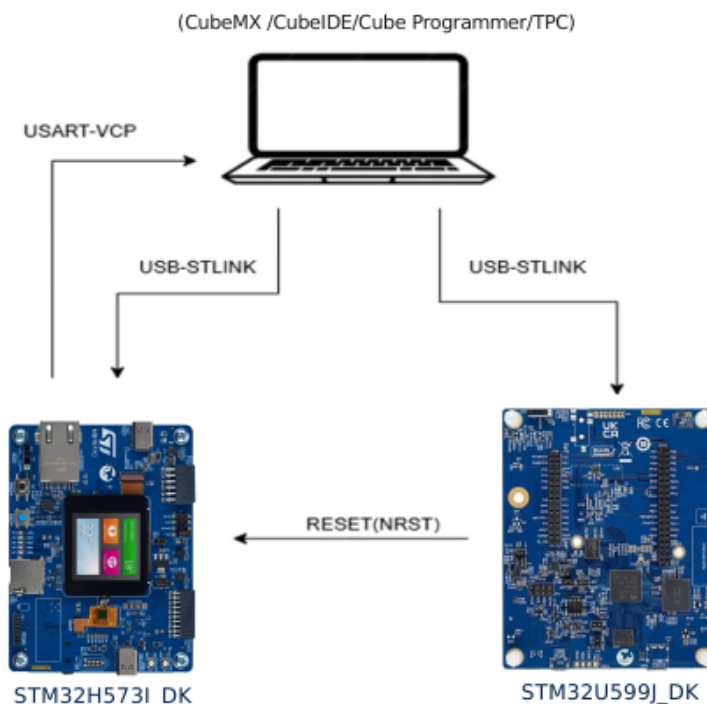


Fig. 3.3 : Architecture des connexions matérielles du banc de validation

3.3 Processus des tests automatiques et architecture globale

La solution développée dans le cadre de ce projet ne se limite pas à l'exécution d'un ensemble de scripts isolés. Elle repose sur une architecture globale pensée pour organiser l'enchaînement des opérations de validation, depuis le lancement du scénario jusqu'à la génération d'un rapport final. Cette architecture permet de structurer le travail de manière modulaire, tout en séparant les responsabilités entre l'interface utilisateur, les mécanismes d'orchestration, les outils STM32, la collecte des preuves et les traitements d'analyse.

3.3.1 Cadre de la conception des tests

La conception du framework est née d'un besoin concret d'industrialiser la validation des scénarios de boot sécurisé. Les campagnes initiales étaient longues, fortement dépendantes des manipulations manuelles et difficiles à reproduire de manière homogène. De plus, la dispersion des logs et l'absence d'un diagnostic structuré rendaient l'analyse des échecs plus complexe.

Pour répondre à ces limites, la solution développée a été pensée comme une chaîne complète de validation, couvrant à la fois l'orchestration des outils STM32, la gestion de la configuration, l'interaction avec le matériel, la collecte des artefacts et la restitution des

résultats. Comme l'illustre la figure 3.4, cette logique repose sur une architecture structurée autour d'une interface utilisateur, d'un orchestrateur central, d'un moteur d'exécution et d'une chaîne de post-traitement menant jusqu'au diagnostic et au reporting final.

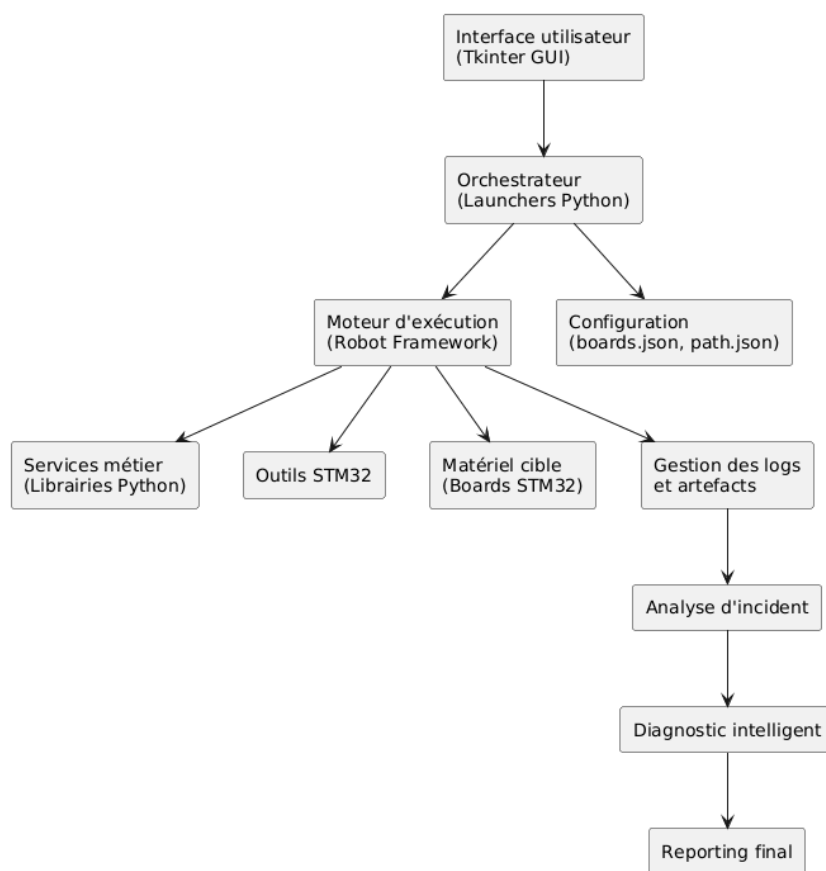


Fig. 3.4 : Logique de conception du framework d'automatisation

3.3.1.1 Architecture d'exécution et déroulement des scénarios automatisés

Le fonctionnement du framework repose sur une chaîne d'exécution structurée, commune à l'ensemble des scénarios de validation du boot sécurisé. Cette organisation permet de guider automatiquement le scénario depuis sa phase de préparation jusqu'au post-traitement final, tout en assurant une bonne séparation entre les étapes de configuration, d'exécution, de validation et d'analyse.

Comme le montre la figure 3.5, le processus débute par la sélection du scénario et de la carte cible, suivie du chargement des paramètres de configuration. À ce niveau, une première distinction apparaît entre les scénarios standards, qui nécessitent une phase de configuration ou de génération de projet avec STM32CubeMX, et les scénarios template, qui reposent sur l'ouverture directe d'un projet existant. Une fois cette étape franchie, le déroulement devient globalement similaire pour l'ensemble des scénarios.

La chaîne de validation se poursuit alors par la compilation des parties *Secure* et *Non-Secure*, la préparation des images sécurisées, le provisioning, la programmation de la cible,

puis la programmation de la carte supervisor afin de déclencher un reset matériel contrôlé. Après le redémarrage de la cible, le framework lance la capture UART et recherche les marqueurs attendus pour conclure sur le succès ou l'échec du boot. En cas de validation, une phase de *full regression* est ensuite exécutée.

Enfin, le scénario se termine par la collecte des logs et des artefacts, la construction de l'incident JSON, l'analyse locale, l'enrichissement par intelligence artificielle et la génération du rapport final. Cette organisation montre que le framework ne se limite pas à l'automatisation d'une séquence d'actions techniques, mais qu'il couvre l'ensemble du cycle de validation, depuis le lancement du scénario jusqu'à l'exploitation des résultats.

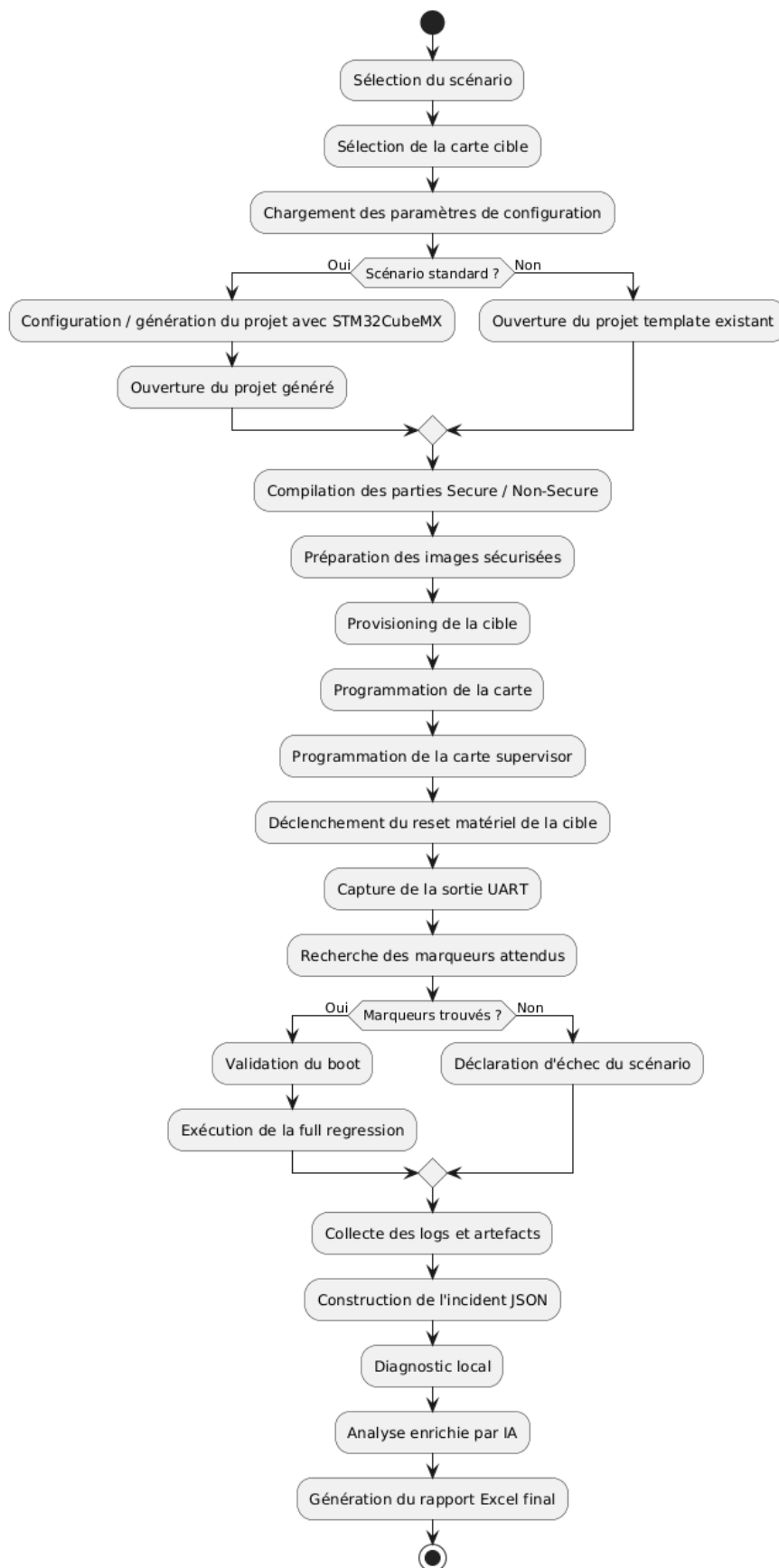


Fig. 3.5 : Déroulement global d'un scénario de validation automatisé

3.4 Mise en œuvre de la solution

Après avoir présenté le cadre général de la solution et son architecture, cette section décrit la manière dont le framework a été concrètement mis en œuvre. L’objectif n’est pas de détailler le code source, mais d’expliquer les grandes briques réalisées, leur rôle dans le processus global et leur contribution à l’automatisation de la validation.

3.4.1 Lancement et pilotage des scénarios

Le point d’entrée principal du framework est une interface graphique développée pour faciliter l’utilisation de la solution. Cette interface a été conçue pour rendre le lancement des scénarios plus accessible, notamment dans un contexte où plusieurs paramètres doivent être définis avant l’exécution : choix de la carte, sélection du scénario, paramètres UART, numéro de série ST-LINK ou encore état du produit.

Au-delà du lancement des scénarios, cette interface offre également un accès direct à certaines commandes utiles pour la préparation ou le contrôle de la cible, comme la lecture des informations de sécurité ou l’exécution d’actions via `STM32_Programmer_CLI`. Elle joue ainsi un rôle central dans l’ergonomie globale de la solution.

Comme le montre la figure 3.6, cette interface regroupe les principales zones de paramétrage du framework : sélection du scénario, choix de la carte, zone de logs en temps réel et raccourcis vers les fichiers générés. Cette organisation permet à l’utilisateur de piloter l’exécution sans dépendre systématiquement de la ligne de commande.

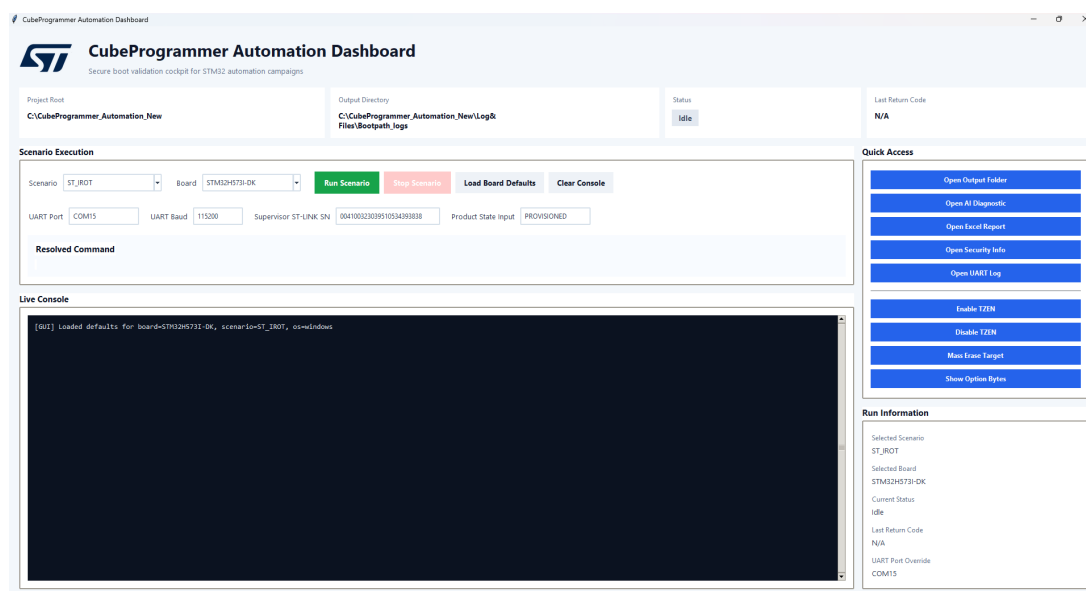


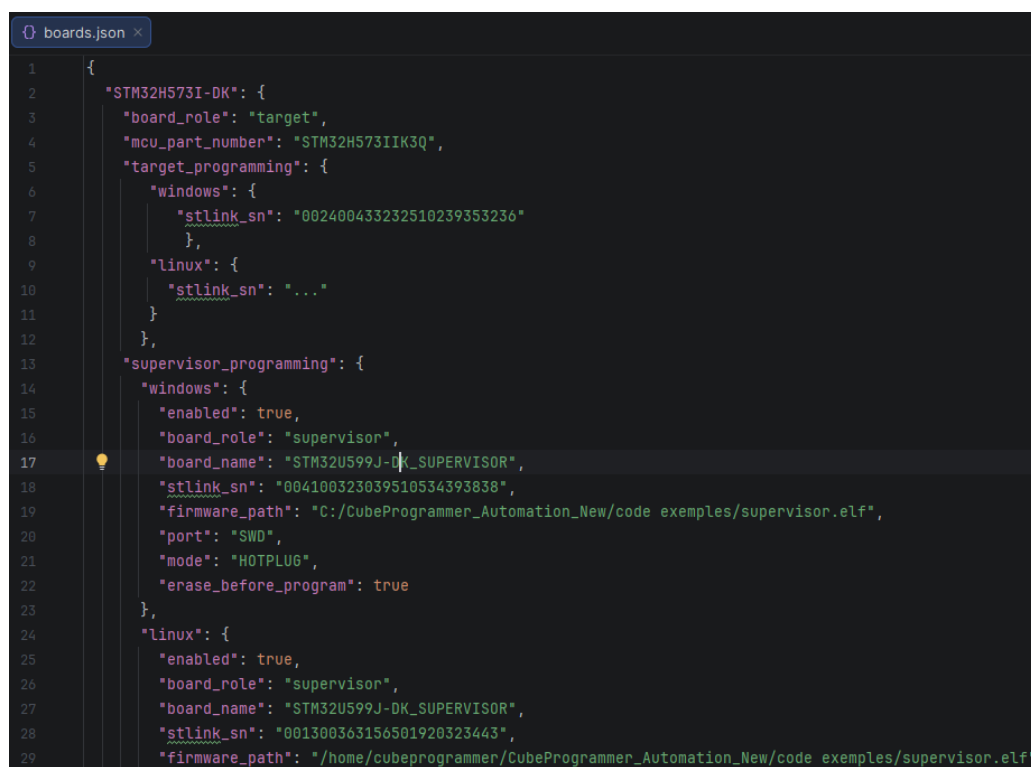
Fig. 3.6 : Interface graphique de pilotage du framework

3.4.1.1 Paramétrage des scénarios et des cartes

Le bon déroulement des scénarios repose sur une configuration centralisée dans des fichiers dédiés. Le fichier `boards.json` contient les informations propres aux cartes supportées, comme les chemins des projets, les paramètres UART, la configuration supervisor ou encore les numéros de série des interfaces ST-LINK. Le fichier `path.json`, quant à lui, centralise les chemins des outils utilisés par le framework.

Cette approche permet de limiter la dispersion des paramètres dans les scripts et rend la solution plus modulaire. Elle facilite également l'ajout de nouvelles cartes ou de nouveaux scénarios, puisque l'évolution du framework peut se faire en grande partie à travers l'adaptation des fichiers de configuration.

La figure 3.7 illustre le rôle de cette couche de configuration. On y observe un exemple de contenu de `boards.json`, montrant comment les informations liées à la cible et à la carte supervisor sont définies pour le framework.



```
1 {
2   "STM32H573I-DK": {
3     "board_role": "target",
4     "mcu_part_number": "STM32H573IIK3Q",
5     "target_programming": {
6       "windows": {
7         "stlink_sn": "002400433232510239353236"
8       },
9       "linux": {
10        "stlink_sn": "..."
11      }
12    },
13    "supervisor_programming": {
14      "windows": {
15        "enabled": true,
16        "board_role": "supervisor",
17        "board_name": "STM32U599J-DK_SUPERVISOR",
18        "stlink_sn": "004100323039510534393838",
19        "firmware_path": "C:/CubeProgrammer_Automation_New/code exemples/supervisor.elf",
20        "port": "SWD",
21        "mode": "HOTPLUG",
22        "erase_before_program": true
23      },
24      "linux": {
25        "enabled": true,
26        "board_role": "supervisor",
27        "board_name": "STM32U599J-DK_SUPERVISOR",
28        "stlink_sn": "001300363156501920323443",
29        "firmware_path": "/home/cubeprogrammer/CubeProgrammer_Automation_New/code exemples/supervisor.elf"
```

Fig. 3.7 : Exemple de fichier de configuration dans le framework

3.4.2 Exécution des scénarios automatisés

Une fois les paramètres définis, le framework déclenche l'exécution du scénario sélectionné. Cette exécution suit une séquence structurée dans laquelle les scripts de lancement, les suites Robot Framework et les outils STM32 interagissent pour reproduire les différentes étapes du flow sécurisé. Même si le contenu exact dépend du scénario choisi, la logique générale reste similaire : préparation du projet, compilation, provisioning, reset,

capture UART, validation et full regression.

Les scénarios `ST_IROT`, `OEM_IROT` et `STIROT_OEMUROT` partagent donc globalement la même chaîne d'exécution. Toutefois, certaines configurations diffèrent selon le flow traité, notamment les chemins de projet, les paramètres de provisioning, les images générées et les marqueurs attendus lors de la validation UART.

3.4.2.1 Préparation du projet et compilation

Dans les scénarios standards, le framework commence par préparer le projet à tester. Cette étape peut inclure la génération du projet avec STM32CubeMX, son ouverture dans STM32CubeIDE, puis la compilation des différentes parties du firmware, notamment les portions *Secure* et *Non-Secure*. Dans les scénarios template, cette préparation est plus directe, puisqu'un projet existant est réutilisé depuis l'environnement STM32CubeFW.

Cette phase est essentielle, car elle conditionne la production des binaires nécessaires aux étapes suivantes. La figure 3.8 illustre cette étape à travers l'ouverture du projet dans STM32CubeIDE et la présence des composants Secure et Non-Secure dans l'environnement de développement.

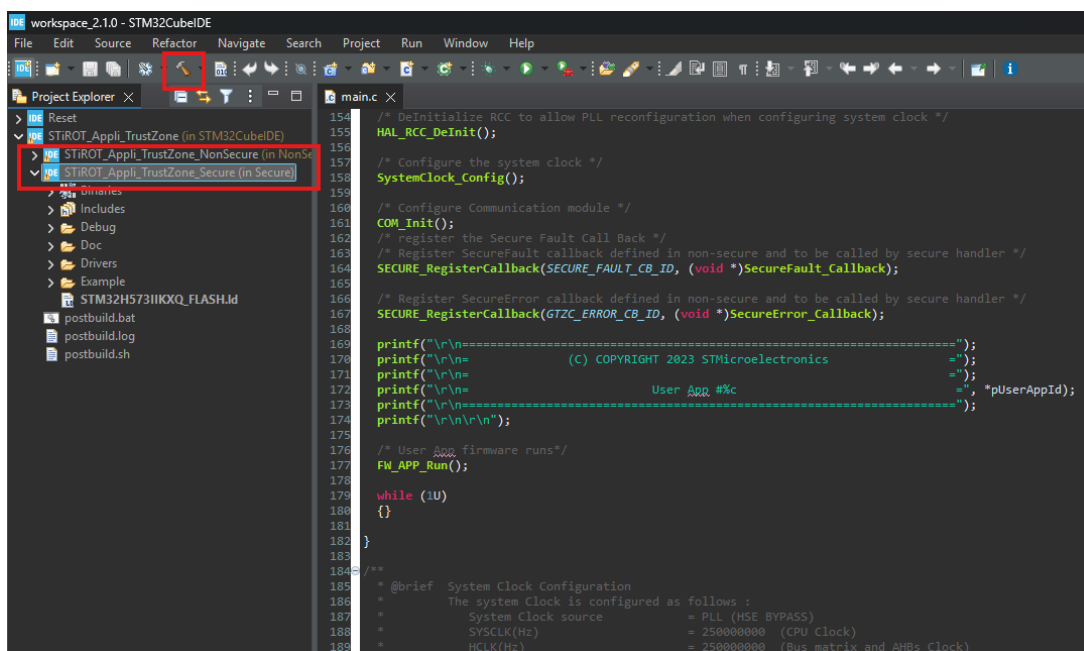


Fig. 3.8 : Préparation du projet et phase de compilation

3.4.2.2 Provisioning et programmation

Une fois les binaires générés, le framework enchaîne avec la phase de provisioning et de programmation. Cette étape consiste à préparer les images nécessaires au scénario de boot sécurisé, à appliquer les paramètres requis par le flow et à programmer les éléments correspondants sur la cible.

Cette séquence est particulièrement sensible, car elle conditionne l'état de sécurité réellement observé au démarrage de la carte. Son automatisation a permis de réduire les manipulations manuelles et d'améliorer l'homogénéité de l'exécution. Comme le montre la figure 3.9, cette phase comprend la génération des images, l'écriture des option bytes, le provisioning des fichiers de configuration et la définition de l'état final du composant.

```

===== Provisioning of STiRoT boot path
===== Application selected through env.bat:
===== C:\Demo_CubeMx\STM32H5_STiRoT_SLed_NSled\ROT_Provisioning\...\
===== Product state must be Open. Execute \ROT_Provisioning\DA\regression.bat if not the case.
=====

* ob_flash_programming script update ...
  Option bytes successfully updated according to STiRoT_Config.xml
  (see "update_ob_setup.log" for details)

Step 2 : Images generation
* Data generation (if Data image is enabled):
  Select STiRoT_Data_Image.xml(Default path is \ROT_Provisioning\STiRoT\Image\STiRoT_Data_Image.xml)
  Generate the data_enc_sign.hex image
  Press any key to continue...

Step 3 : Provisioning
* BOOT0 pin should be disconnected from VDD:
  (STM32H573I-DK: set Sw1 to position 0)
  Press any key to continue...

* Programming the option bytes and flashing the images ...
  Successful option bytes programming and images flashing
  (see "ob_flash_programming.log" for details)

* Define product state value
  [ OPEN | PROVISIONED | TZ-CLOSED | CLOSED | LOCKED ]: CLOSED

* Setting the product state PROVISIONING

* Provisioning the .obk files ...
  Successful obk provisioning
  (see "obkey_provisioning.log" for details)

* Setting the final product state CLOSED

=====
===== The board is correctly configured.
===== Power off/on the board to start the application.
=====

```

Fig. 3.9 : Phase de provisioning et de programmation

3.4.2.3 Reset matériel via la carte supervisor

Après la phase de provisioning, il est nécessaire de redémarrer la cible dans des conditions fiables afin de capturer correctement son comportement au boot. Pour cela, la carte STM32U599J-DK est utilisée comme carte supervisor et programmée avec un firmware dédié capable de générer un reset matériel sur la cible.

Le principe repose sur une liaison directe entre une sortie GPIO de la carte supervisor et la broche NRST de la carte cible, avec une masse commune entre les deux cartes. Comme le montre la figure 3.10, le reset est déclenché par la carte STM32U5 en forçant la ligne NRST de la STM32H5 à l'état bas, ce qui provoque un redémarrage matériel contrôlé de la cible.

Cette approche permet de mieux maîtriser la séquence de redémarrage et d'améliorer la synchronisation avec la capture UART. Elle apporte ainsi une solution plus robuste et plus directe que des méthodes plus anciennes reposant sur des mécanismes indirects ou moins stables.

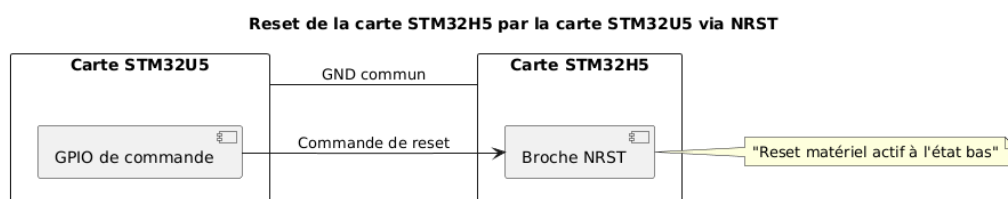


Fig. 3.10 : Principe du reset matériel de la carte cible via la carte supervisor

3.4.2.4 Capture UART et validation du boot

Après le reset matériel, le framework lance une capture UART pendant une durée déterminée. Cette capture constitue la preuve principale de validation du boot, car le succès du scénario repose sur la présence de marqueurs attendus dans la sortie série. Selon le scénario exécuté, ces marqueurs peuvent correspondre à des messages de boot, à des traces de mise à jour ou à l’affichage d’un menu template.

La figure 3.11 présente un exemple de trace UART obtenue lors d’un scénario template. On y retrouve les éléments attendus, tels que le message de copyright, l’identification de l’application utilisateur et les entrées du menu principal, ce qui permet de conclure au bon déroulement du boot.

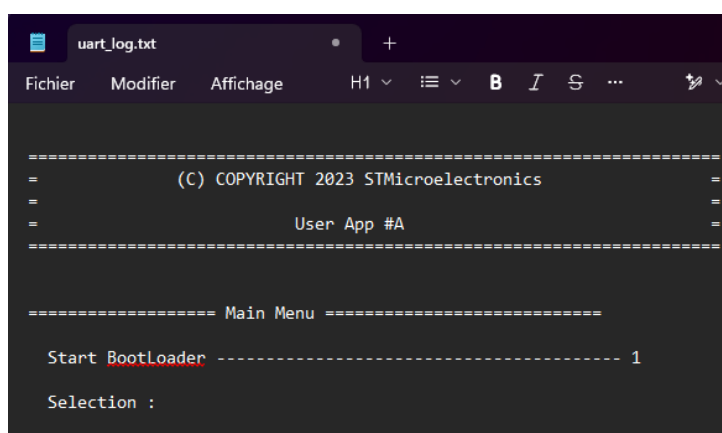
A screenshot of a text editor window titled 'uart_log.txt'. The window shows a UART log capture with the following content: a header line '===== (C) COPYRIGHT 2023 STMicroelectronics =====', followed by 'User App #A' on a new line, another header line '===== Main Menu =====', and then 'Start BootLoader ----- 1' and 'Selection :'. The text is displayed in a monospaced font with some color coding (e.g., 'BootLoader' in red).

Fig. 3.11 : Capture UART

3.4.2.5 Exécution de la full regression

La full regression constitue la dernière étape fonctionnelle du scénario. Son objectif est de compléter la validation après le boot, en exécutant une série de contrôles supplémentaires sur la carte cible. Dans le cadre du projet, cette étape a posé une difficulté particulière lorsque plusieurs interfaces ST-LINK étaient connectées simultanément, car les scripts de régression ne savaient pas toujours distinguer clairement la carte cible de la carte supervisor.

Pour résoudre ce problème, une amélioration a été introduite à travers le ciblage explicite de la cible via `TARGET_STLINK_SN` et la génération d’un script de régression adapté. Cette correction a renforcé la robustesse de l’exécution finale et a permis d’éviter les ambiguïtés de ciblage.

3.4.3 Gestion des artefacts et post-traitement

L’un des apports majeurs du framework développé réside dans sa capacité à centraliser non seulement l’exécution des scénarios, mais aussi l’ensemble des artefacts produits à l’issue d’un run. Cette centralisation est essentielle pour la traçabilité, car elle permet

de conserver à la fois les résultats d'exécution, les preuves UART, les informations de sécurité, les logs système et les éléments de diagnostic.

Au-delà de cette collecte, une véritable logique de post-traitement a été mise en place. Elle transforme les résultats bruts en un incident structuré, en recommandations locales, puis en un diagnostic enrichi. Comme le montre la figure 3.12, cette chaîne de post-traitement relie directement l'exécution du scénario à la production d'un rapport final exploitable.

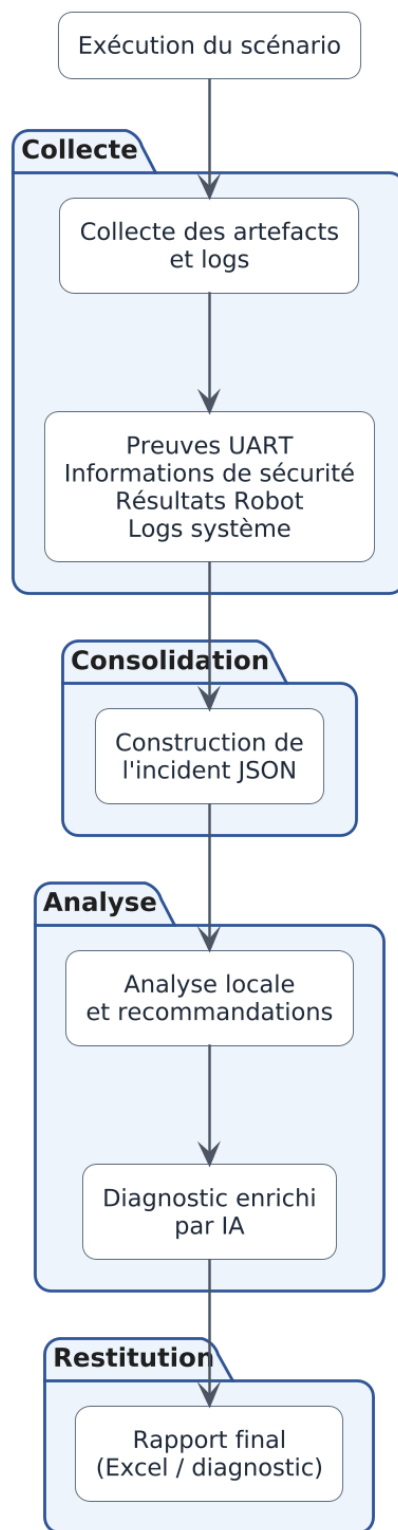


Fig. 3.12 : Chaîne de post-traitement après l'exécution d'un scénario

3.4.3.1 Collecte des logs et preuves de validation

À la fin de chaque exécution, le framework regroupe plusieurs fichiers dans un répertoire dédié aux logs. On y retrouve notamment les rapports Robot Framework, les traces UART, les fichiers de sortie standard et d'erreur, les logs liés à la supervisor, les fichiers

JSON d'incident, ainsi que les rapports générés après traitement.

La figure 3.13 illustre l'organisation de ces artefacts dans le dossier `Bootpath_logs`. Cette structure centralisée facilite la consultation des résultats et la conservation d'un historique exploitable des campagnes exécutées.

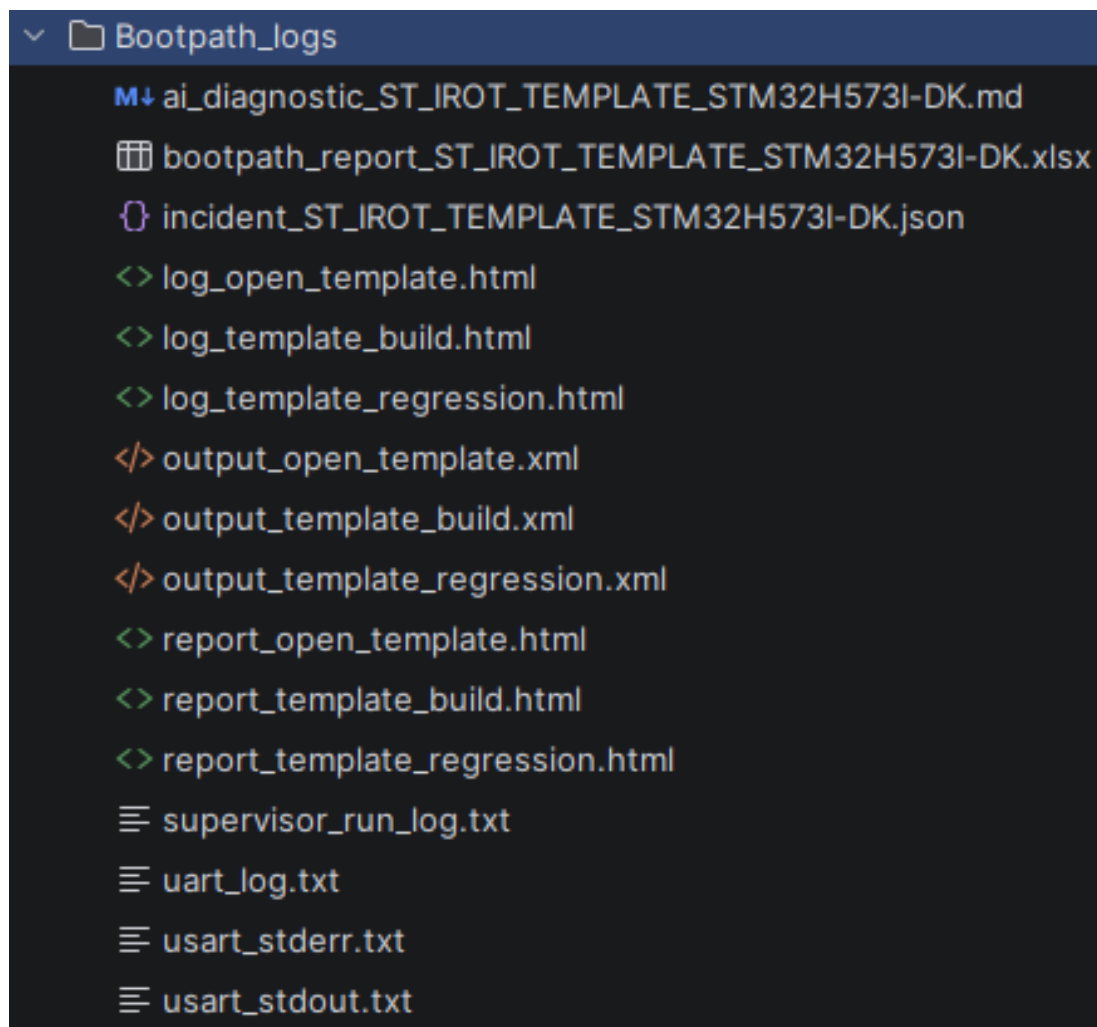
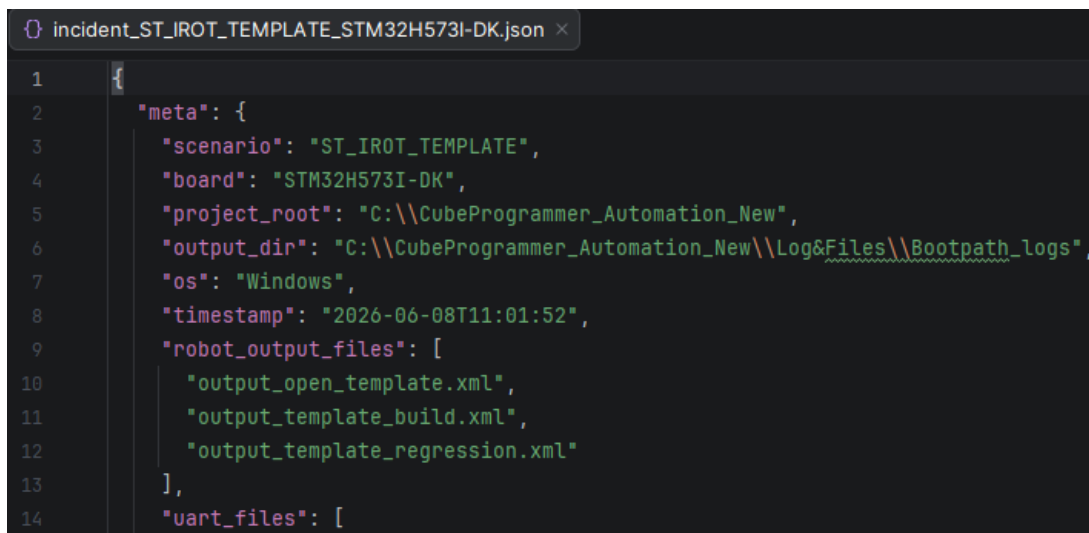


Fig. 3.13 : Organisation des artefacts et des logs générés

3.4.3.2 Construction de l'incident JSON

Le framework ne construit pas son diagnostic directement à partir de chaque log isolé. Il passe d'abord par une étape de consolidation, au cours de laquelle les principales informations du run sont regroupées dans un incident JSON unique. Cet incident contient les résultats Robot, la sortie UART, les informations de sécurité, les erreurs éventuelles ainsi que certaines métadonnées utiles à l'analyse.

La figure 3.14 montre un exemple de fichier d'incident généré. Elle met en évidence la structuration des informations essentielles du scénario dans un format unique, qui sert ensuite de base à la chaîne de diagnostic.



```
1 {
2   "meta": {
3     "scenario": "ST_IROT_TEMPLATE",
4     "board": "STM32H573I-DK",
5     "project_root": "C:\\CubeProgrammer_Automation_New",
6     "output_dir": "C:\\CubeProgrammer_Automation_New\\Log&Files\\Bootpath_logs",
7     "os": "Windows",
8     "timestamp": "2026-06-08T11:01:52",
9     "robot_output_files": [
10      "output_open_template.xml",
11      "output_template_build.xml",
12      "output_template_regression.xml"
13    ],
14    "uart_files": [
```

Fig. 3.14 : Construction de l'incident JSON à partir des artefacts du run

3.4.3.3 Diagnostic local et enrichissement par IA

Une fois l'incident construit, une première couche d'analyse locale permet de classer certaines erreurs, d'interpréter les états UART et de produire des recommandations simples. Ensuite, le framework enrichit cette analyse grâce à un appel à un modèle de langage, afin de générer une synthèse plus lisible et mieux contextualisée.

La figure 3.15 montre un exemple de diagnostic généré après cette phase de post-traitement. On y distingue les informations de contexte, les recommandations locales, les incidents similaires ainsi que la synthèse IA produite à partir de l'incident.

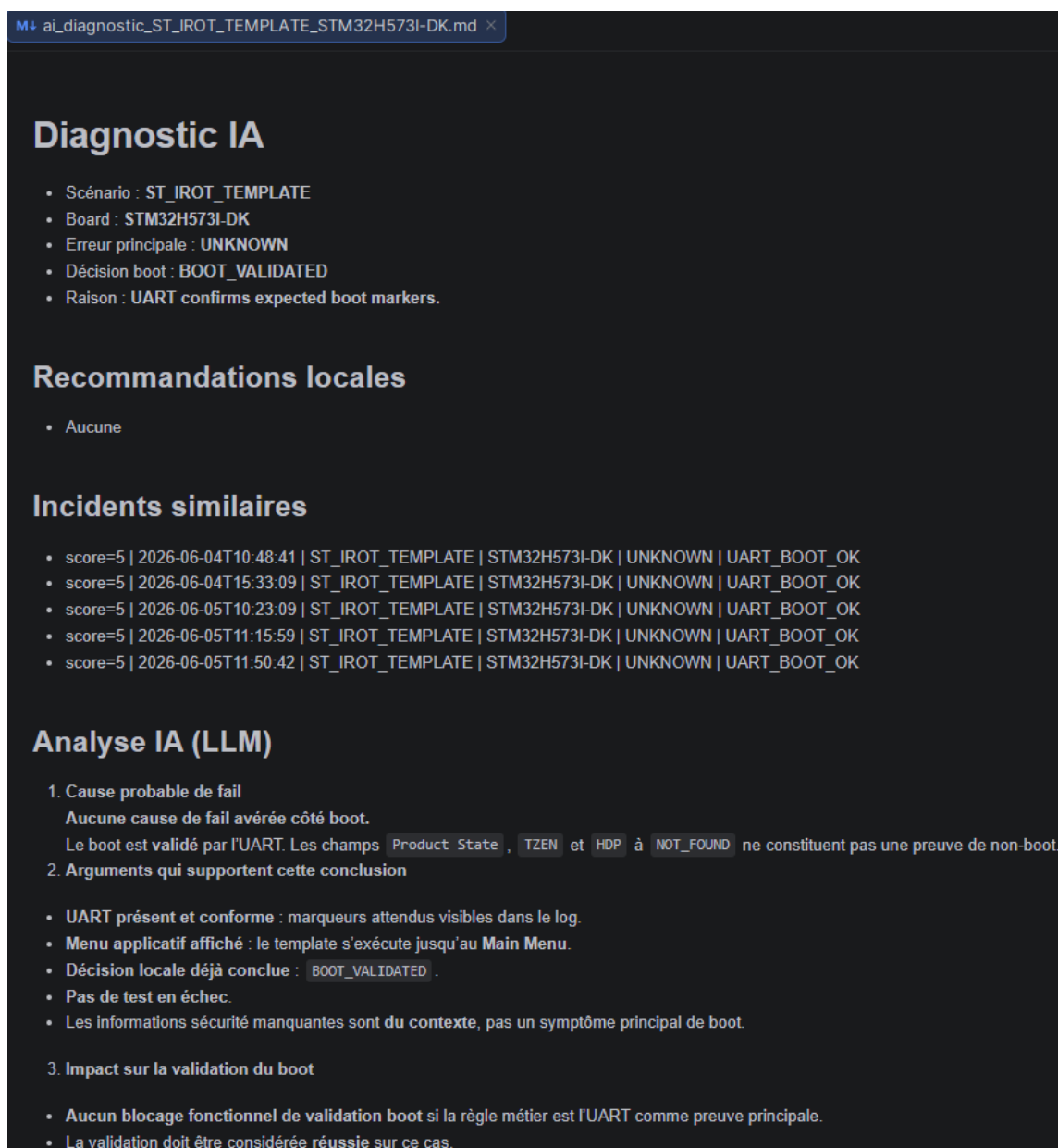


Fig. 3.15 : Rapport IA généré

Afin d'illustrer la capacité du framework à diagnostiquer un cas d'échec, la figure 3.16 présente un exemple de diagnostic généré à la suite d'un run sur le scénario ST_IROT n'ayant pas permis de prouver le bon déroulement du boot. Dans ce cas, l'erreur principale identifiée est UART_EMPTY, et la décision de boot est fixée à BOOT_NOT_PROVEN, avec une confiance qualifiée de faible (LOW). La raison associée précise que la sortie UART n'a pas fourni de preuve concluante du démarrage attendu. Les recommandations locales orientent alors l'utilisateur vers trois vérifications simples : le port UART configuré, l'état de boot réel de la carte cible, et la cohérence du provisioning avec l'image générée. L'analyse enrichie par le modèle de langage complète ensuite ce diagnostic en identifiant la cause probable du fail — l'absence réelle de preuve de boot sur l'UART, plutôt qu'un problème d'interface graphique — et en s'appuyant sur des éléments de preuve concrets : fichier

UART vide, sortie COM indiquant zéro octet écrit et absence des marqueurs de boot attendus. Le rapport IA propose enfin une liste d’actions correctives priorisées, allant de la vérification du chemin UART/VCP à la validation du provisioning et du firmware programmé. Cette structuration permet à l’ingénieur d’orienter rapidement son investigation, sans avoir à relire manuellement l’ensemble des logs bruts.

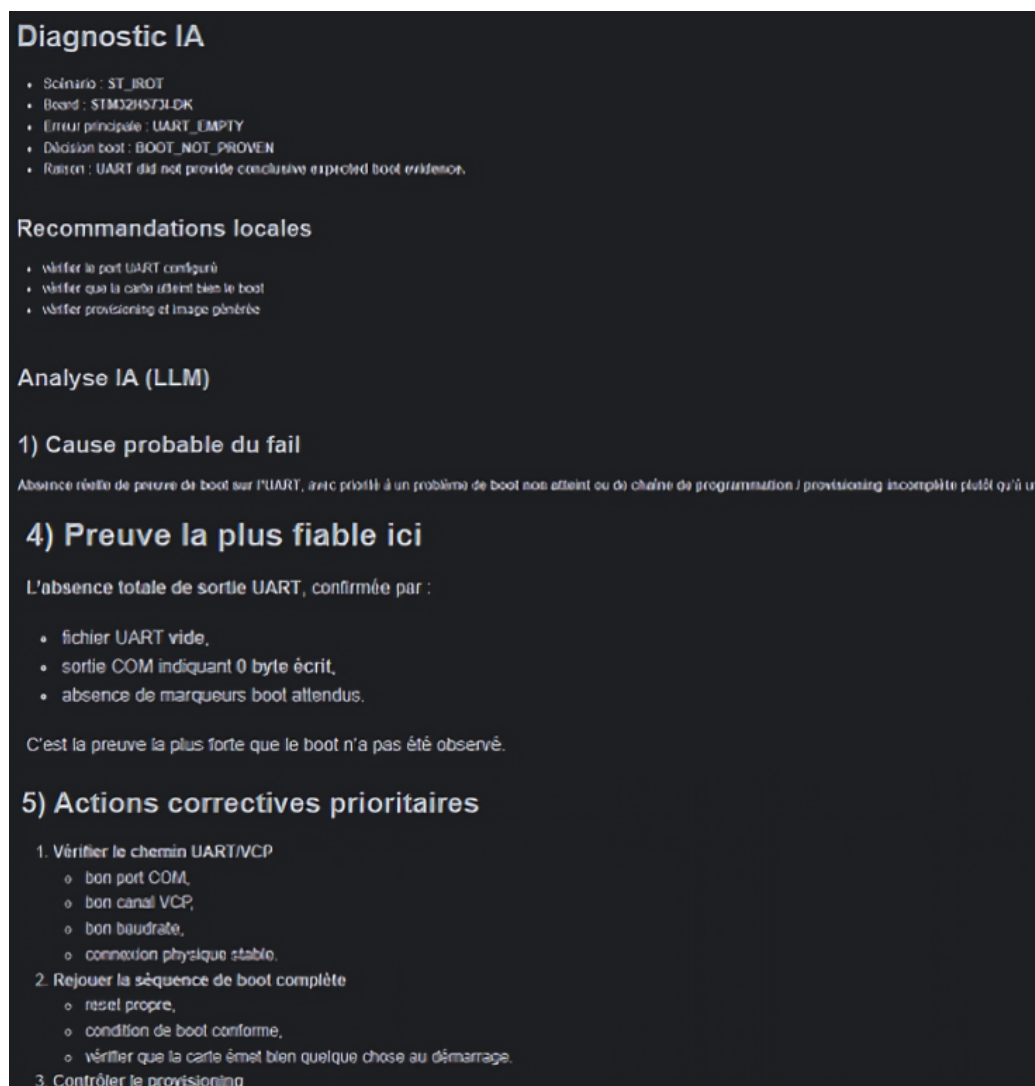


Fig. 3.16 : Exemple de diagnostic IA pour un run échoué (scénario ST_IROT)

3.4.3.4 Génération du rapport Excel

La dernière étape du post-traitement consiste à produire un rapport Excel de synthèse. Ce rapport a été conçu pour offrir une lecture rapide et structurée d’un run, en regroupant les informations essentielles dans différentes feuilles : résumé général, synthèse du diagnostic, preuves UART et détail des tests exécutés.

La figure 3.17 présente la structure générale du rapport généré par le framework. On y retrouve notamment les champs de synthèse du run, la décision de validation du boot ainsi qu’un tableau récapitulatif des étapes exécutées lors du scénario.

	A	B			
1	Field	Value			
2	Scenario	ST_IROT_TEMPLATE			
3	Board	STM32H573I-DK			
4	OS	Windows			
5	Timestamp	2026-06-08T11:01:52			
6	Tests	4			
7	PASS	4			
8	FAIL	0			
9	NOT_EXECUTED	0			
10	UART State	UART_BOOT_OK			
11	Boot Decision	BOOT_VALIDATED			
12	Decision Reason	UART confirms expected boot markers.			
13	Confidence	HIGH			
	Scenario	Board	Phase	Test	Status
1	ST_IROT_TEMPLATE	STM32H573I-DK	OPEN_TEMPLATE	Open Existing STiROT Template Project In CubeIDE	PASS
2	ST_IROT_TEMPLATE	STM32H573I-DK	TEMPLATE_BUILD	Build Template Secure And NonSecure Projects	PASS
3	ST_IROT_TEMPLATE	STM32H573I-DK	TEMPLATE_BUILD	TEST BOOT	PASS
4	ST_IROT_TEMPLATE	STM32H573I-DK	TEMPLATE_REGRESSIO N	Execute Full Regression	PASS

Fig. 3.17 : Rapport Excel généré par le framework après post-traitement

La figure 3.18 illustre la structure du rapport Excel généré pour ce même run en échec. On y observe que, sur les 13 tests exécutés, 12 sont marqués PASS et 1 est marqué FAIL, ce qui traduit une exécution presque complète du scénario mais un échec localisé au niveau de la preuve de boot. Le champ **UART State** indique **UART_EMPTY**, et la **Boot Decision** est mise en évidence en rouge avec la valeur **BOOT_NOT_PROVEN**, accompagnée du motif "UART did not provide conclusive expected boot evidence". Cette cohérence entre le rapport Excel et le diagnostic IA garantit que l'utilisateur dispose d'une vision homogène du résultat, qu'il consulte la synthèse rapide du rapport ou l'analyse détaillée du diagnostic. Les images correspondantes sont fournies sous les noms `ia_echec.png` et `excel_echec.png` dans le répertoire `images`.

	A	B
1	Field	Value
2	Scenario	ST_IROT
3	Board	STM32H573I-DK
4	OS	Windows
5	Timestamp	2026-06-25T11:18:39
6	Tests	
7	PASS	
8	FAIL	
9	NOT_EXECUTED	
10	UART State	UART_EMPTY
11	Boot Decision	BOOT_NOT_PROVEN
12	Decision Reason	UART did not provide conclusive expected boot evidence.
13	Confidence	LOW

Fig. 3.18 : Extrait du rapport Excel pour un run échoué

3.5 Résultats obtenus et discussion

Le framework développé prend en charge plusieurs scénarios de boot sécurisé, correspondant aux familles de flows étudiées au cours du projet. Parmi eux figurent les scénarios standards ST_IROT, OEM_IROT et STIROT_OEMUROT, ainsi que les scénarios template ST_IROT_TEMPLATE et OEM_IROT_TEMPLATE. Cette diversité montre que la solution n'a pas été conçue pour un seul chemin d'exécution, mais comme un cadre suffisamment générique pour couvrir plusieurs configurations de validation.

La figure 3.17 illustre un exemple de comparaison de validation d'un scénario ST_IROT entre une méthode manuelle et l'exécution automatique du framework. Sur ce cas, le temps de validation passe d'environ 15 minutes en mode manuel à 8 minutes en mode automatique, ce qui représente un gain de temps significatif sur la campagne de vérification.

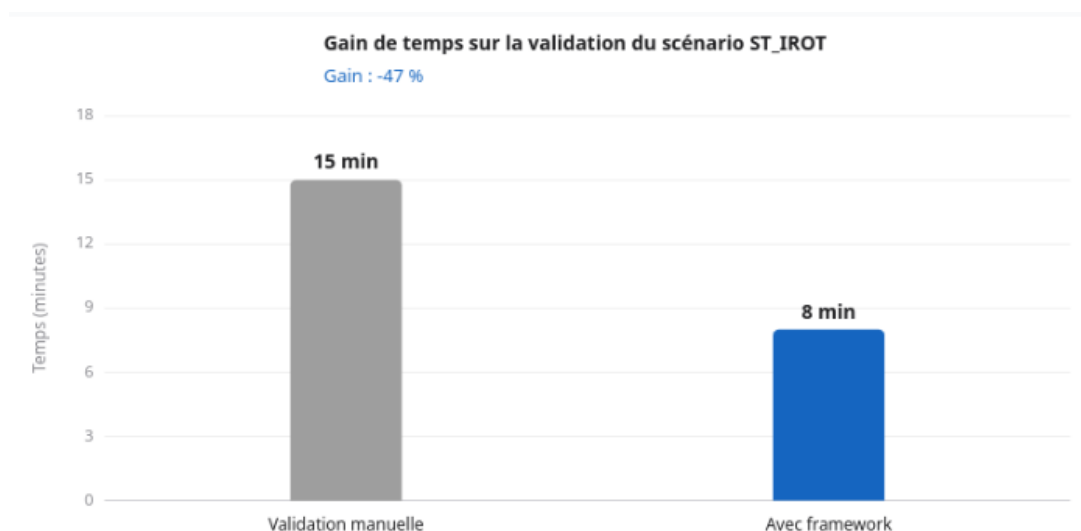


Fig. 3.19 : Comparaison du temps de validation d'un scénario ST_IROT : validation manuelle vs validation automatique

Les résultats obtenus montrent que le framework est exploitable pour les scénarios actuellement couverts. La stabilité des flows template a été améliorée, la logique de reset matériel a renforcé la maîtrise du démarrage de la cible, et la prise en compte explicite du numéro de série ST-LINK pour la cible a permis de lever une difficulté importante dans les campagnes de full regression.

D'un point de vue plus global, les apports du framework peuvent être résumés autour de trois axes principaux. Le premier concerne la réduction des manipulations manuelles, notamment lors des séquences répétitives impliquant plusieurs outils STM32. Le deuxième concerne la traçabilité, puisque les preuves de validation, les logs et les résultats sont désormais centralisés dans une même chaîne de traitement. Enfin, le troisième apport réside dans l'intégration d'une logique de post-traitement capable d'aller au-delà d'un simple résultat *pass/fail* pour fournir une analyse plus riche et plus exploitable.

3.6 Considérations finales et limitations

Malgré les avancées réalisées, la solution développée présente encore certaines limites qu'il est important de mentionner. La première concerne la dépendance résiduelle à l'automatisation graphique. Même si plusieurs opérations ont été fiabilisées, les interactions GUI restent plus sensibles que les appels directs en ligne de commande. Elles dépendent notamment du focus de la fenêtre, de la résolution de l'écran, du timing d'affichage et de l'état de certains éléments visuels.

D'autres limites concernent l'environnement d'exécution lui-même. Certains scénarios restent sensibles aux versions des outils STM32 installés, à l'organisation du poste de test ou à la présence simultanée de plusieurs interfaces de programmation. Enfin, même

si l'analyse diagnostique a été enrichie, certaines informations de sécurité peuvent encore être absentes ou incomplètes selon le contexte d'exécution.

Ces limites ne remettent pas en cause l'intérêt du framework, mais elles montrent que la solution reste perfectible et qu'elle peut encore évoluer vers une automatisation plus robuste et plus indépendante de l'environnement graphique.

3.7 Conclusion

Dans ce chapitre, nous avons présenté la réalisation concrète du framework d'automatisation développé dans le cadre de ce projet, ainsi que son évaluation globale. Nous avons d'abord décrit la configuration matérielle utilisée, en montrant le rôle des cartes cibles, de la carte supervisor et des interfaces de connexion. Nous avons ensuite détaillé l'architecture générale de la solution et le processus de tests automatiques qui structure l'exécution des scénarios de validation.

La mise en œuvre du framework a ensuite été présentée à travers ses principales composantes fonctionnelles : lancement des scénarios, préparation du projet, provisioning, reset matériel, capture UART, full regression, collecte des artefacts, construction d'incidents, diagnostic et génération de rapports. Enfin, nous avons discuté les résultats obtenus ainsi que les principales limites qui subsistent.

L'ensemble de ces résultats montre que la solution développée répond de manière concrète au besoin d'industrialisation de la validation des scénarios de boot sécurisé STM32. Elle constitue une base solide pour poursuivre l'amélioration des campagnes de test et renforcer encore davantage la robustesse de la chaîne de validation.

Conclusion générale

La validation des mécanismes de sécurité dans les systèmes embarqués représente aujourd'hui un enjeu essentiel, en particulier dans le contexte des microcontrôleurs modernes où la protection du code, l'intégrité du démarrage et la maîtrise du cycle de vie du composant occupent une place centrale. C'est dans cette perspective que s'inscrit le projet réalisé au sein de STMicroelectronics, dont l'objectif principal était de concevoir et de mettre en œuvre un framework d'automatisation dédié à la validation de scénarios de boot sécurisé sur cartes STM32.

Le travail mené a abouti au développement d'une solution complète, structurée autour d'une chaîne de validation capable d'orchestrer l'exécution des scénarios, de piloter différents outils de l'écosystème STM32, de gérer les interactions avec le matériel, de capturer les preuves d'exécution et de produire des résultats exploitables. Le framework repose notamment sur Python, Robot Framework, STM32CubeMX, STM32CubeIDE, STM32CubeProgrammer et STM32TrustedPackageCreator, qui ont été intégrés dans une architecture modulaire et évolutive.

L'un des apports majeurs du projet réside dans la centralisation des différentes étapes du processus de validation. La solution développée permet de couvrir l'ensemble du cycle d'un scénario, depuis le lancement par l'utilisateur jusqu'à la génération d'un rapport final, en passant par la compilation, le provisioning, le reset matériel de la cible, la capture UART, l'exécution de la régression et le post-traitement des résultats. Dans cette architecture, la sortie UART occupe une place centrale, puisqu'elle constitue la preuve principale de validation du bon déroulement du boot. Cette approche a permis de s'appuyer sur un indicateur directement lié au comportement réel observé sur la cible.

Le projet a également intégré une dimension avancée d'analyse et d'assistance au diagnostic. À partir des artefacts produits lors des campagnes de test, le framework construit un incident JSON structuré, puis applique une première logique de diagnostic local avant d'enrichir cette analyse à l'aide d'un modèle de langage. Cette approche permet non seulement de mieux interpréter les incidents, mais aussi de transformer des logs techniques en résultats plus lisibles et plus exploitables pour les phases d'investigation.

Les résultats obtenus montrent une amélioration concrète de la répétabilité des validations, une réduction importante des manipulations manuelles et un meilleur niveau de traçabilité sur l'ensemble des campagnes de test. La solution développée apporte ainsi une réelle valeur ajoutée dans un contexte industriel, en dépassant la logique d'exécution

automatique pour proposer une véritable chaîne de validation orientée preuve, analyse et reporting.

Malgré les progrès réalisés, certaines limites subsistent, notamment la dépendance partielle à l'automatisation graphique, la sensibilité à l'environnement du poste de test et la fragilité éventuelle de certains scénarios face aux changements d'interface ou de configuration. Ces éléments ouvrent naturellement la voie à plusieurs perspectives d'amélioration, telles que la réduction de la dépendance aux interfaces GUI, le renforcement des appels directs en ligne de commande, l'enrichissement du reporting ou encore l'extension du framework à de nouveaux scénarios et à de nouvelles cibles matérielles.

Au-delà des résultats techniques, ce projet a constitué une expérience particulièrement formatrice sur les plans scientifique, technique et professionnel. Il m'a permis de consolider mes compétences dans les systèmes embarqués sécurisés, l'automatisation des tests, l'orchestration logicielle, l'exploitation des outils STM32 et l'analyse assistée par intelligence artificielle. Il m'a également offert l'opportunité de travailler dans un environnement industriel exigeant, où la rigueur, l'adaptabilité et la qualité de l'ingénierie jouent un rôle déterminant dans la réussite des projets.

Bibliographie

- [1] STMicroelectronics, “Stmicroelectronics company presentation,” 2025, available online. [Online]. Available : <https://www.st.com/content/dam/aboutus/who-we-are/pdf/st-company-presentation-en.pdf>
- [2] —, “Stmicroelectronics tunisia,” 2025, corporate information and local R&D activities.
- [3] Wikipedia contributors, “Stmicroelectronics — wikipedia, the free encyclopedia,” 2025, available online. [Online]. Available : <https://fr.wikipedia.org/wiki/STMicroelectronics>
- [4] STMicroelectronics, *STM32H5 Series Reference Manual*, 2025, reference manual.
- [5] —, *STM32U5 Series Reference Manual (RM0456)*, 2025, reference manual. [Online]. Available : https://www.st.com/resource/en/reference_manual/rm0456-stm32u5-series-armbased-32bit-mcus-stmicroelectronics.pdf
- [6] —, *STM32H7 Series Reference Manual*, 2025, reference manual.
- [7] —, *STM32H573I-DK Discovery Kit Data Brief / User Manual*, 2025, board documentation.
- [8] —, *STM32H7S78-DK Discovery Kit Data Brief / User Manual*, 2025, board documentation.
- [9] —, *STM32U599J-DK Discovery Kit Data Brief / User Manual*, 2025, board documentation.
- [10] —, “Stm32cubemx,” 2025, official tool documentation. [Online]. Available : <https://www.st.com/en/development-tools/stm32cubemx.html>
- [11] —, “Stm32cubeide,” 2025, official integrated development environment. [Online]. Available : <https://www.st.com/en/development-tools/stm32cubeide.html>
- [12] —, “Stm32cubeprogrammer,” 2025, official programming tool. [Online]. Available : <https://www.st.com/en/development-tools/stm32cubeprog.html>

- [13] —, “Stm32trustedpackagecreator,” 2025, trusted package creation tool used in secure boot flows.
- [14] —, “Trustzone-enabled stm32 security documentation,” 2025, application notes and security documentation.
- [15] STMicroelectronics Community, “What option bytes in stm32 are and how to use them,” 2025, available online. [Online]. Available : <https://community.st.com/t5/stm32-mcus/what-option-bytes-in-stm32-are-and-how-to-use-them/ta-p/49451>
- [16] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge University Press, 2016.
- [17] R. Black, E. van Veenendaal, and D. Graham, *Foundations of Software Testing : ISTQB Certification*, 4th ed. Cengage Learning, 2018.
- [18] Test Academy, “Boîte noire vs boîte blanche,” 2025, available online. [Online]. Available : <https://testacademy.fr/boite-noire-vs-boite-blanche/>
- [19] All4Test, “Différents tests logiciels,” 2025, available online. [Online]. Available : <https://www.all4test.fr/differents-test-logiciel/>
- [20] ARM Ltd., *ARM Architecture Reference Manual*. ARM, 2020.
- [21] Arrow Electronics, “Engineering basics : What is a microcontroller?” 2025, available online. [Online]. Available : <https://www.arrow.com/en/research-and-events/articles/engineering-basics-what-is-a-microcontroller>
- [22] Passion Electronique, “Liaisons série : Uart, i2c et spi,” 2025, available online. [Online]. Available : <https://passionelectronique.fr/liaisons-series-uart-i2c-spi/>
- [23] CircuitBread, “How usb works : Communication protocol,” 2025, available online. [Online]. Available : <https://www.circuitbread.com/tutorials/how-usb-works-communication-protocol-part-2>
- [24] STMicroelectronics, “Stm32 microcontroller debug toolbox,” 2025, application note. [Online]. Available : https://www.st.com/resource/en/application_note/an4989-stm32-microcontroller-debug-toolbox-stmicroelectronics.pdf
- [25] Python Software Foundation, “Python documentation,” 2025, official documentation. [Online]. Available : <https://docs.python.org/3/>
- [26] —, “Tkinter — python interface to tcl/tk,” 2025, official documentation. [Online]. Available : <https://docs.python.org/3/library/tk.html>

- [27] C. Liechti and contributors, “pyserial documentation,” 2025, official documentation. [Online]. Available : <https://pyserial.readthedocs.io/>
- [28] openpyxl contributors, “openpyxl documentation,” 2025, official documentation. [Online]. Available : <https://openpyxl.readthedocs.io/>
- [29] Requests contributors, “Requests : Http for humans,” 2025, official documentation. [Online]. Available : <https://requests.readthedocs.io/>
- [30] Robot Framework Foundation, “Robot framework user guide,” 2025, official documentation. [Online]. Available : <https://robotframework.org/>
- [31] SikuliX, “Sikulix automation tool,” 2025, official documentation. [Online]. Available : <https://sikulix.github.io/>
- [32] PyAutoGUI contributors, “Pyautogui documentation,” 2025, official documentation. [Online]. Available : <https://pyautogui.readthedocs.io/>
- [33] Pyperclip contributors, “Pyperclip documentation,” 2025, clipboard module for Python. [Online]. Available : <https://pyperclip.readthedocs.io/>
- [34] Git SCM, “Git documentation,” 2025, official documentation. [Online]. Available : <https://git-scm.com/doc>
- [35] CodeProject, “Git branching model,” 2025, available online. [Online]. Available : <https://www.codeproject.com/Articles/1165512/GIT-Branching-Model>
- [36] JetBrains, “Pycharm official documentation,” 2025, available online. [Online]. Available : <https://www.jetbrains.com/help/pycharm/>
- [37] InetProcess, “Gestion de projet avec la méthode waterfall,” 2025, available online. [Online]. Available : <https://www.inetprocess.com/fr/blog/gestion-de-projet-avec-la-methode-waterfall-ou-modele-en-cascade>
- [38] Sooyoos, “Méthodologie scrum,” 2025, available online. [Online]. Available : <https://www.sooyoos.com/publication/scrum/>
- [39] Bocasay, “Méthode kanban dans les projets de développement informatique,” 2025, available online. [Online]. Available : <https://www.bocasay.com/fr/methode-kanban-projets-developpement-informatique/>
- [40] Jenkins, “Jenkins documentation,” 2025, official documentation. [Online]. Available : <https://www.jenkins.io/doc/>
- [41] R. Bommasani *et al.*, “On the opportunities and risks of foundation models,” arXiv preprint arXiv :2108.07258, 2021.

-
- [42] A. Vaswani, N. Shazeer, N. Parmar *et al.*, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [43] DAIR.AI, “Prompt engineering guide,” 2025, available online. [Online]. Available : <https://www.promptingguide.ai/>

Conception et développement d'un framework d'automatisation pour la validation du boot sécurisé

Rapport de Stage de PFE DGT-ENIM, 2026

RÉSUMÉ

Ce rapport présente la conception et le développement d'un framework d'automatisation destiné à la validation de scénarios de boot sécurisé sur microcontrôleurs STM32 au sein de STMicroelectronics. La solution développée s'appuie sur Python, Robot Framework et les outils de l'écosystème STM32 afin d'automatiser la préparation des projets, la compilation, le provisioning, le reset matériel, la capture UART et la génération de rapports. Le framework intègre également une chaîne de post-traitement fondée sur la construction d'un incident JSON, un diagnostic local et un enrichissement par intelligence artificielle. Les résultats obtenus montrent une amélioration de la répétabilité des tests, une meilleure centralisation des preuves de validation et une capacité de diagnostic plus structurée.

Mots clés : boot sécurisé, STM32, automatisation des tests, Robot Framework, UART, diagnostic IA.

ABSTRACT

This report presents the design and development of an automation framework dedicated to the validation of secure boot scenarios for STM32 microcontrollers at STMicroelectronics. The proposed solution relies on Python, Robot Framework and STM32 ecosystem tools to automate project preparation, build, provisioning, hardware reset, UART capture and reporting. The framework also includes a post-processing chain based on incident JSON generation, local diagnosis and AI-assisted enrichment. The obtained results show improved test repeatability, better centralization of validation evidence and a more structured diagnostic capability.

Keywords : secure boot, STM32, test automation, Robot Framework, UART, AI-assisted diagnosis.